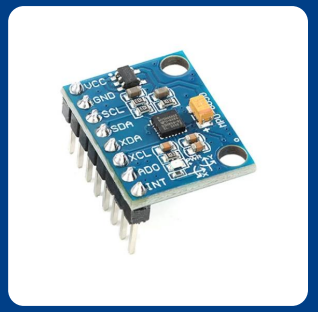
SENSOR PRACTICE · C10B

传感器大课













显示 · 感知 · 执行 · 姿态 · 通信

SMART CAR · SENSORS & SERIAL

小车传感器及串口通讯

讲透 C10B 的五类外设与通讯

实验中心　仲玉芳

一 · 显示与执行　二 · 姿态感知　三 · 串口通讯

讲授	平台	日期
3 堂课	C10B · STM32F103	2026.10

THREE SESSIONS

三讲讲什么

第一讲

显示与执行

- OLED：显存与时序
- 超声波：捕获计时
- 蜂鸣器与舵机
- 代码走读 + 实验

第二讲

姿态感知

- 六轴与 MEMS 原理
- 软件 I²C 与 DMP
- 四元数与漂移
- 零偏校准与实验

第三讲

串口通讯

- 电气标准与一帧
- HAL 与三种收发
- BCC 与两套报文
- 排错与联调

CHAPTER 01

01

显示与执行

把状态变成字，把电信号变成动作

本讲内容

- OLED：屏、显存与时序、初始化
- 超声波：测距原理与实现方式
- 蜂鸣器：种类与控制
- 舵机：原理、接线、控制

本讲要get什么

能点亮一块 OLED

交互内容怎么显示出来、汉字显示怎么实现

能算出距离

看懂 Echo 宽度，理解为什么用输入捕获

能让蜂鸣器响、舵机转

有源无源的判据、20 ms 脉宽对应角度

能迁移到新模块

每个模块都按“器件—原理—代码”走一遍

OLED 为什么会发光

电流型发光器件

空穴与电子在发光层复合，激子退激发光

亮度正比于电流

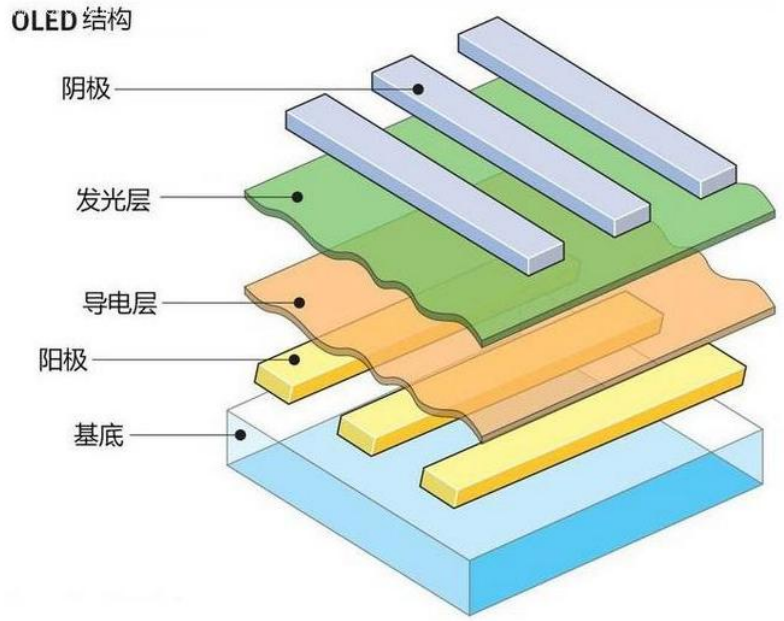
不需要背光，所以黑得下去、也省电

有机薄膜多层结构

阳极 / 传输层 / 发光层 / 阴极，驱动电压低

比 LCD 轻薄

对比度极高、响应快、可做柔性，代价是寿命



记住一点就够：它自己发光，所以每个像素只有“亮”和“灭”两种状态

OLED 与 LCD 差在哪

两种显示技术的差别

对比项	OLED	LCD
发光方式	像素自己发光	需要背光
黑色表现	不发光，纯黑	背光漏光，灰
响应速度	微秒级	毫秒级
功耗	按亮像素算	背光常亮
寿命	偏短，蓝光衰减快	长
成本	高	低

> 模块只有 0.04 W，就是因为黑的地方真的不耗电

模块参数：八个数

C10B 使用的 OLED 模块参数

参数	指标
分辨率	128 × 64 点阵
可视角度	大于 160°
功耗	正常显示约 0.04 W
供电范围	**直流 3.3** – 5 V
工作温度	−30 ~ 70 ℃（工业级）
体积	29.2 × 27.5 mm
通信接口	4 线 SPI（另有 2 线 I2C 版本）
驱动芯片	SSD1306



模块自带 4.7 kΩ 上拉和升压电路，接上 3.3 V 就能用，不用外挂元件

六根引脚，各管一件事

OLED 模块引脚与 C10B 接线

引脚	作用	C10B 引脚
DC	数据 / 命令切换	PB3
RES	复位，低电平有效	PB4
SDA	数据线（SPI 的 MOSI）	PB5
SCL	时钟线	PC14
VCC	电源正 3.3 V	3.3 V
GND	电源负，必须共地	GND



DC 是关键：同一根数据线，靠它区分“这是命令”还是“这是要显示的点”

原点在左上角



X 向右，Y 向下

和数学习惯相反，写坐标时最容易错

128 列 × 64 行

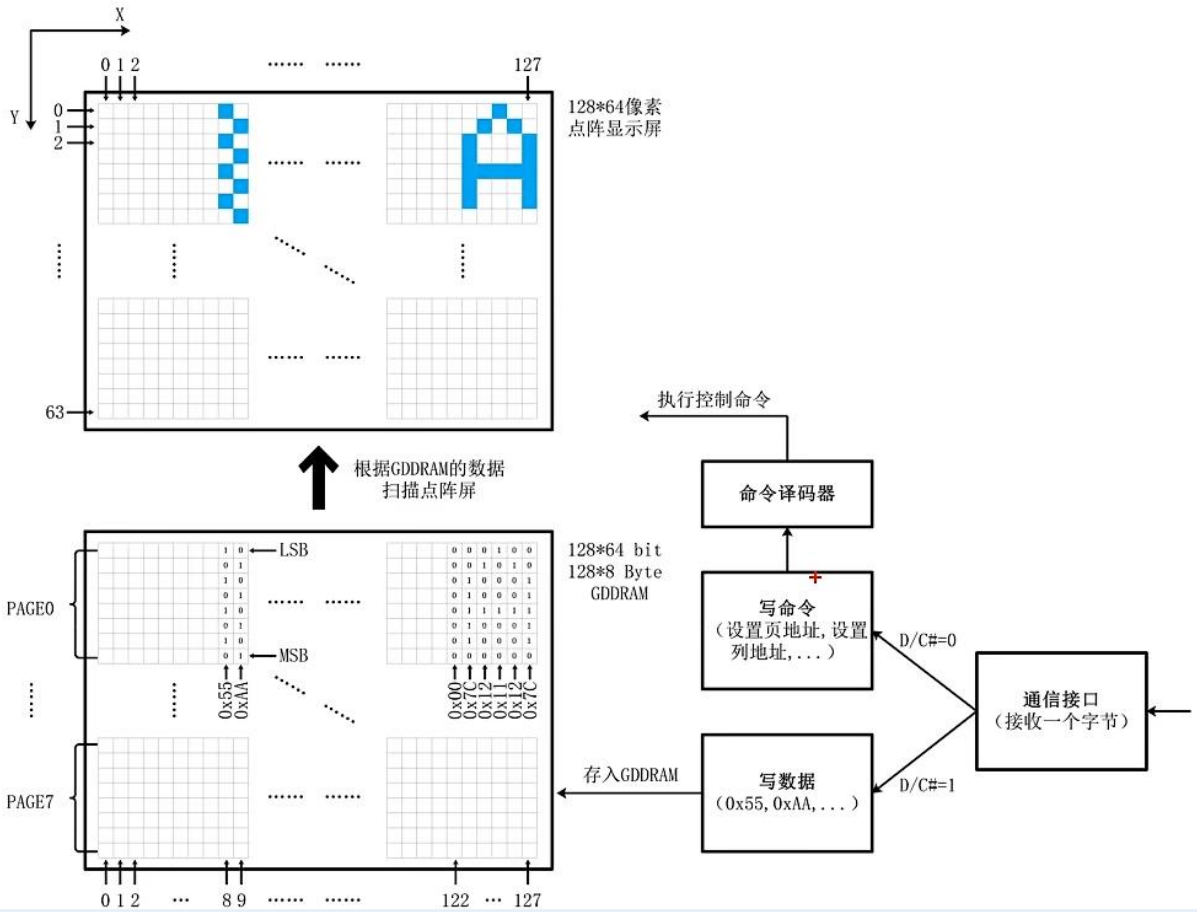
一个像素一个点，没有灰度

按页组织

64 行拆成 8 页，每页 8 行

GRAM 与硬件显存

显示原理框图



要显示的数据写进单片机的 GRAM，屏幕上是硬件显存，中间靠 OLED_Refresh() 同步

页寻址：8 行算一页

64 行拆成 8 页

每页 8 行，page 0 ~ page 7

一个字节管 8 个竖点

最低位在上，最高位在下

写满 128 列就是一页

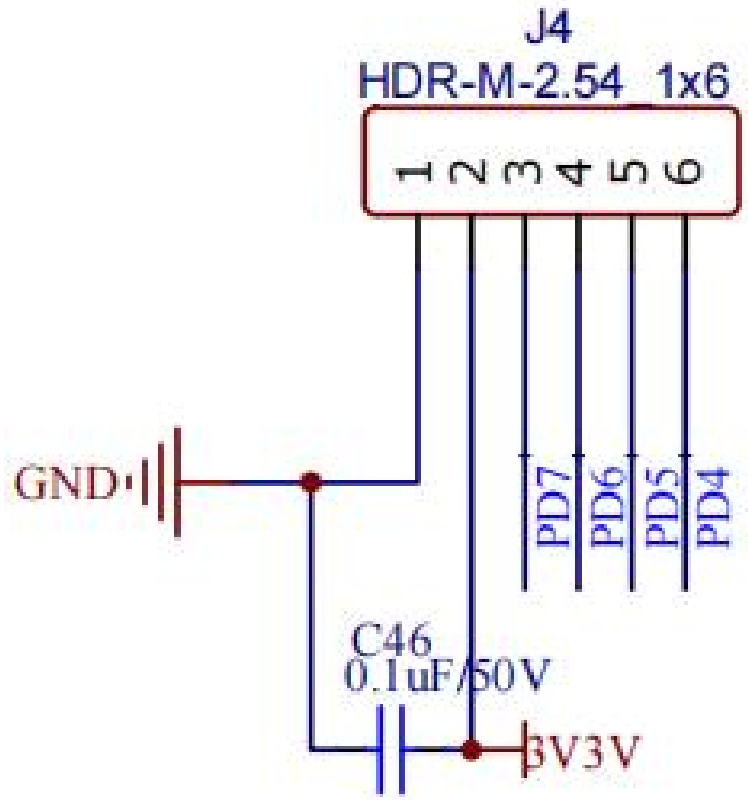
写指针自动右移，不用手动算

显存一共 1 KB

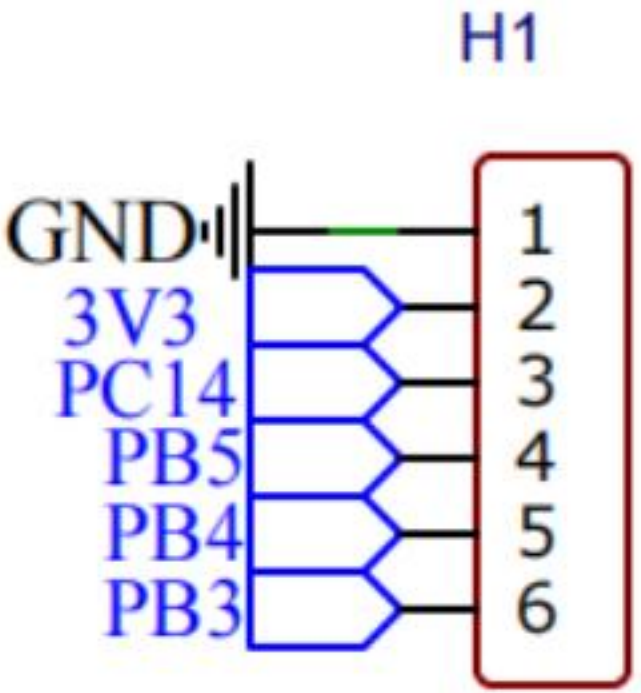
128 × 64 ÷ 8 = 1024 字节

> 所以“清屏”就是把这 1024 字节全写 0，再刷新一次

电路接口

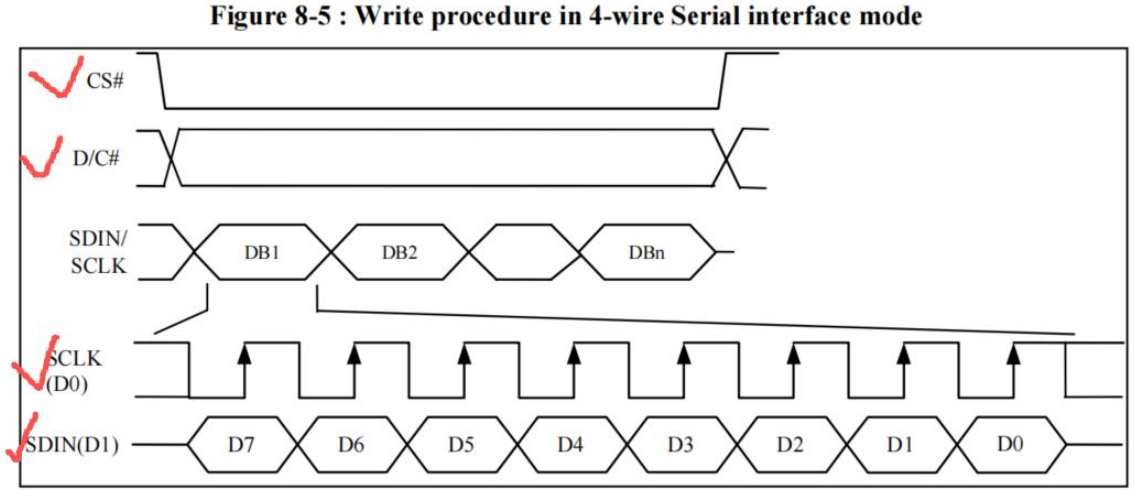


C50C OLED模块接口



C10B OLED模块接口

写一个字节的时序



写字节时序

见手册P17-18

先定 DC

DC = 1 写数据，DC = 0 写命令

上升沿移入

先把数据摆稳，再给时钟沿

MSB 先发

左移一位走一个 bit，八次一字节

OLED的原子操作（bsp_oled.c）

```
void OLED_WR_Byte(uint8_t dat, uint8_t cmd)
{
    uint8_t i;
    if (cmd) OLED_DC_Set();       //写数据
    else     OLED_DC_Clr();       //写命令
    for (i = 0; i < 8; i++) {
        OLED_SCLK_Clr();          // 先拉低
        if (dat & 0x80) OLED_SDIN_Set();
        else            OLED_SDIN_Clr();
        OLED_SCLK_Set();          // 上升沿移入
        dat <<= 1;
    }
    OLED_DC_Set();
}
```

写命令、写数据、刷屏

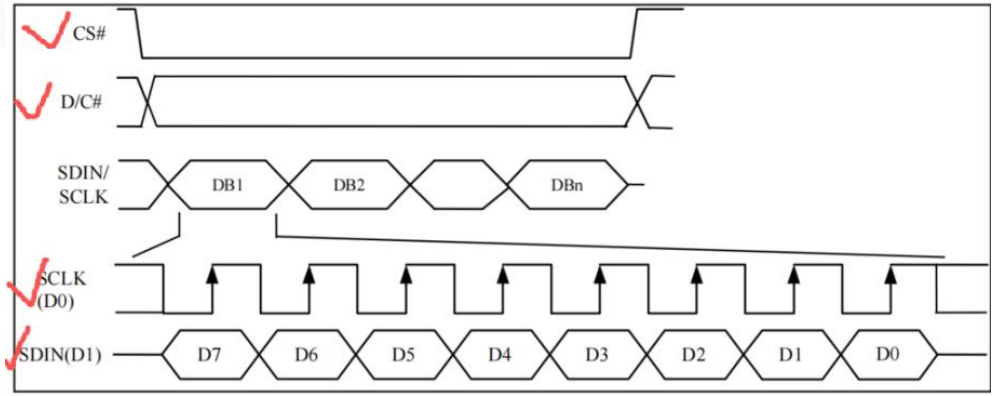
全都循环调用它

看懂这几行

剩下只剩“发什么”的问题

SCLK 必须先低后高

顺序反了首位就丢



OLED正常操作流程

整体两大阶段

1）初始化流程（上电仅执行 1 次）

2）根据需要绘图 + 刷新显示流程（循环执行）

初始化：前四条

01 **关显示　0xAE**
上电先关屏，避免花屏被人看见

02 **时钟分频　0xD5 + 0x80**
设振荡频率与分频比，决定刷新速度

03 **复用行数　0xA8 + 0x3F**
0x3F = 63，即 64 行 COM

04 **偏移与起始行　0xD3 / 0x40**
画面上下微调，起始行从第 0 行开始

这四条配上电复位，是“屏亮之前”的准备动作

初始化：后四条

05 **COM 硬件配置　0xDA + 0x12**
适配 128×64 屏的引脚排布，写错会上下反

06 **预充电周期　0xD9 + 0xF1**
影响亮度和残影

07 **VCOMH　0xDB + 0x30**
消隐电平，决定黑色有多黑

08 **电荷泵　0x8D + 0x14**
开内部升压，不开屏就永远不亮

最后还有三步：设寻址模式 → 0xAF 开显示 → 清屏，见下一页

初始化：最后三步

09 **设寻址模式　0x20 + 0x02**
页寻址，写满一页自动换页

10 **开显示　0xAF**
到这一步屏幕才亮，前面都是黑屏

11 **清屏 + 刷新**
显存是随机值，不清屏就是雪花

12 **自检：写一行字**
能显示说明时序、寻址、字库都对

13 条指令按顺序发，少一条都可能黑屏，多一条不会坏

绘图与刷新，是两个阶段

绘图只改内存

DrawPoint / DrawLine / ShowString 改的是 GRAM

刷新才动硬件

OLED_Refresh() 一次推完 1024 字节

为什么要分两步

SPI 一位一位发，攒够一屏再发才不打断主循环

寻址模式默认页寻址；开显示 0xAF 之后必须先清屏，否则显存是随机值

绘图函数族：都只改内存

bsp_oled.c 里常用的绘图接口

函数	参数	作用
OLED_Clear	无	1024 字节全写 0
OLED_DrawPoint	x, y, color	画一个点，越界自动丢弃
OLED_DrawLine	x1,y1,x2,y2	两点之间打点，斜线也能画
OLED_ShowChar	x, y, ch, size	6×8 或 8×16 点阵
OLED_ShowString	x, y, str, size	逐字调用 ShowChar
OLED_Refresh	无	把 GRAM 整块推到屏幕

> 常犯的错：调完 DrawPoint 就以为屏上有了，其实还差一次 Refresh

引脚在 main.h 里改，不用动驱动

```
#define OLED_DC_Pin         GPIO_PIN_3
#define OLED_DC_GPIO_Port   GPIOB
#define OLED_RES_Pin        GPIO_PIN_4
#define OLED_RES_GPIO_Port  GPIOB
#define OLED_SDA_Pin        GPIO_PIN_5
#define OLED_SDA_GPIO_Port  GPIOB
#define OLED_SCL_Pin        GPIO_PIN_14
#define OLED_SCL_GPIO_Port  GPIOC
```

C50C 用 PD4~PD7

同一套驱动，换宏就能跑

四线 SPI，没有 CS

所以只能挂一个 SPI 屏

换板子只改这 8 行

驱动层完全复用

这些脚都是输出脚，配置成推挽输出，无上下拉，见gpio.c

屏不带字库，字模要自己取

内置两套 ASCII 点阵

12×6 与 16×8，够显示数字和英文，参见oledfont.h和bsp_oled.c

汉字要自己取模

PCtoLCD2002：阴码、逐列式、顺向

取模参数必须对

行列式 / 逐行式选错，字就是花的

作为人机交互够用

显示 Pitch / Roll / Yaw / 距离即可

> 工程里点阵放在 oledfont.h，加字就是把新数组塞进去

取模四步，一条都不能错

01 **选字体与字号**
常用宋体 12/16 号，越小越糊

02 **设参数**
阴码、逐列式、顺向、C51 格式

03 **点阵生成**
宽高要是 8 的倍数，否则要补边

04 **粘进 oledfont.h**
按 size 分表，索引是字符 ASCII

参数选错的表现很统一：字是花的、或者只剩半边，不是代码坏了

OLED 上最容易踩的四个坑

屏不亮

90% 是电荷泵没开（0x8D + 0x14），或者换屏试

花屏或上下反

COM 配置 0xDA 与段重映射 0xA1 配错

显示全黑但背光正常

忘了 0xAF，或者清屏没执行

刷新慢到卡主循环

一边算一边刷，应该算完再整体刷新

实验一：点亮第一行字

01 **正确安装模块**
VCC/GND/SCL/SDA/RES/DC，别插反

02 **确认电压**
先量 3.3 V 与 GND，再看复位脚

03 **烧写并看现象**
黑屏查电荷泵，花屏查 COM 配置

04 **显示一行字**
ShowString + Refresh，屏上有了就过关

字符串显示正常后，可以尝试怎么实现在屏幕上显示中文

HC-SR04：超声波传感器

HC-SR04 主要技术参数

参数	指标
工作电压	DC 5 V
静态电流	小于 2 mA
输出电平	高 5 V / 低 0 V
感应角度	不大于 15°
探测距离	2 cm – 400 cm（2 cm 以内是盲区）
测距精度	约 ±3 mm
接线	VCC / TRIG / ECHO / GND



盲区 2 cm：太近反而测不到，装车时要留出这段距离

什么叫超声波

频率高于 20 kHz

人耳上限 20 kHz，所以听不见

方向性好、穿透力强

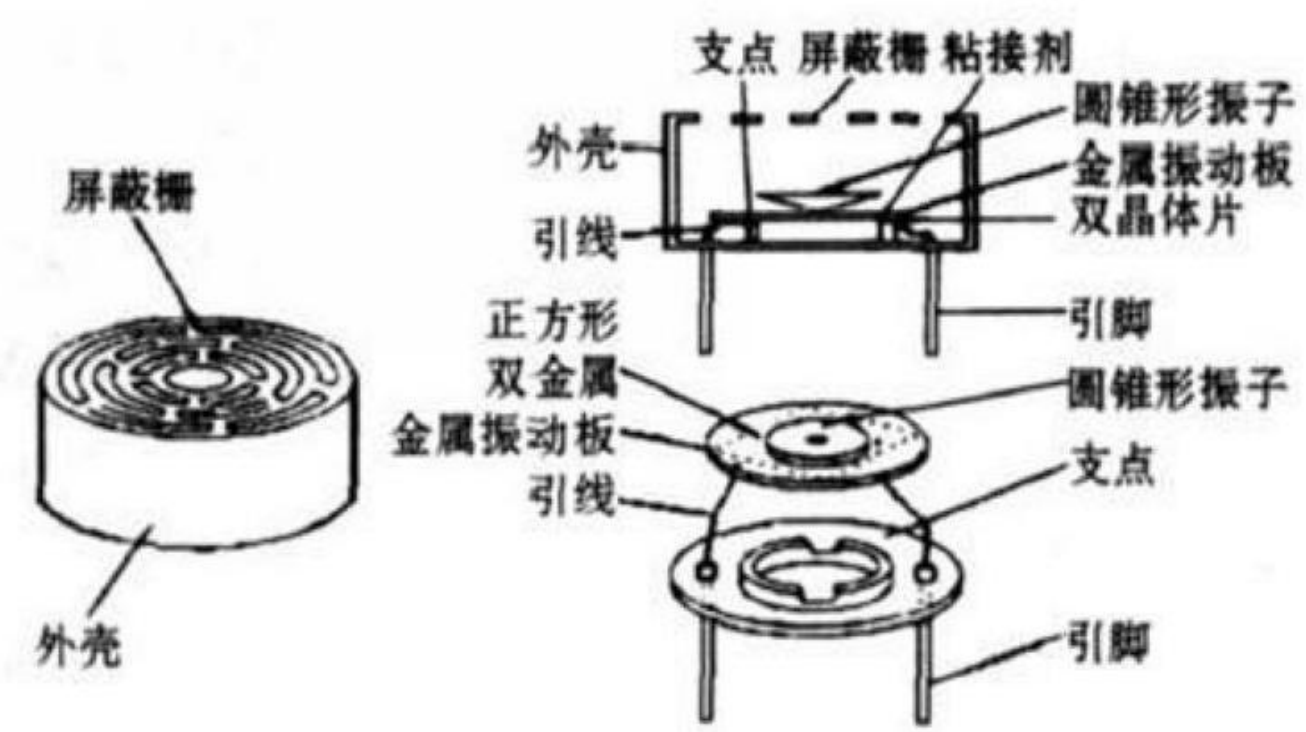
能量集中，适合测距与探伤

空气中约 340 m/s

温度每升 1 ℃，声速约涨 0.6 m/s

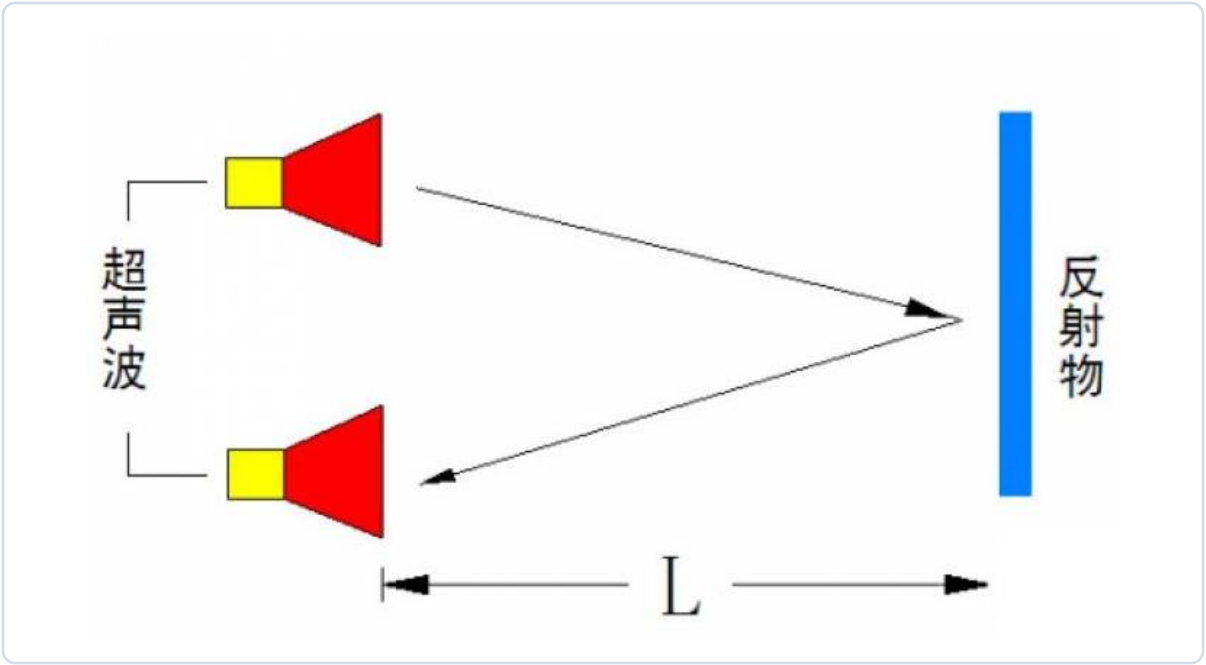
常见 40 kHz

HC-SR04 用的就是这个频点



顺带一个工程常识：声速随温度变，冬天和夏天测同一距离，读数会差 1% 左右

超声波测距原理



发射 → 反射 → 接收

测的是往返总时间 t

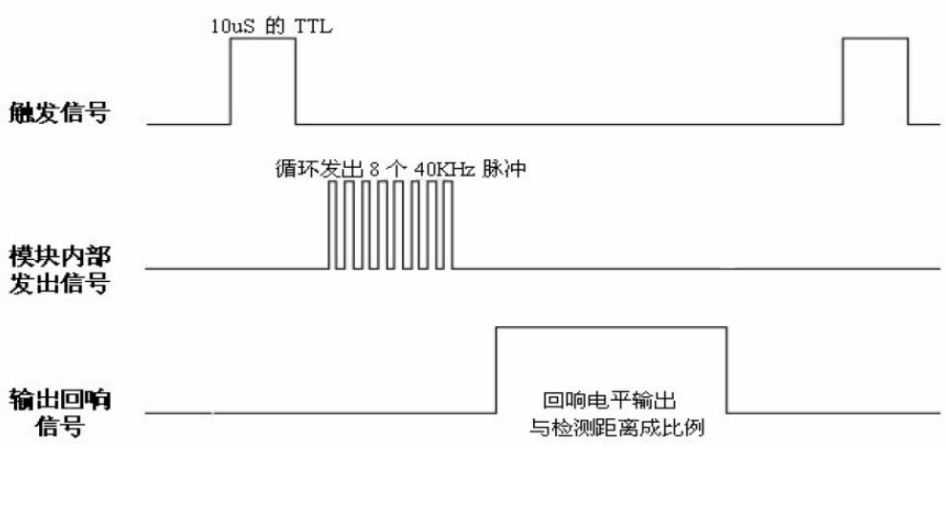
L = v•t / 2

v ≈ 340 m/s，别忘除2

只测时间

精度取决于计时精度，不取决于声速

测距实现



测距时序

TRIG ≥ 10 µs

工程上给 15 µs 更保险

ECHO 变高

高电平宽度 = 往返时间

间隔 ≥ 60 ms

回波没散完就发，两发会串

超声波测距的实现方式

超声波测距的四种实现方式

实现方式	实时性	精度	CPU占用	外设需求	适用场景
软件 GPIO 轮询	极差，全程阻塞	一般	极高	仅普通 GPIO	教学演示
外部中断 + 定时器	中等	高	中高（频繁中断）	GPIO + 独立定时器	单路、简易小车
单通道输入捕获	**优秀**	**极高**	**低**	**1 路定时器捕获通道**	**正式工程首选**
门控 / 双通道捕获	最优	最高	极低	定时器双通道 / 门控资源	多路同步、低功耗

C10B 用的是第三种：单通道输入捕获，TIM2 的 CH2

C10B上超声波硬件接口

PA1（echo）为什么能和模块直连?

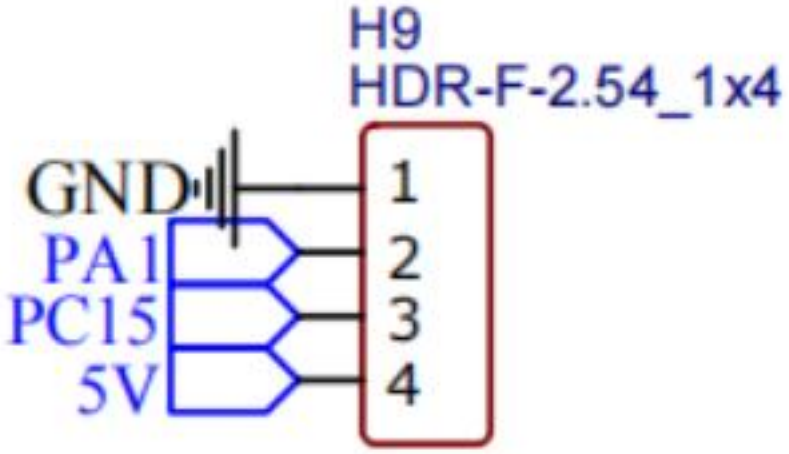
F103 标注 FT 的 IO 可容忍 5 V 输入

稳妥做法仍要分压

1 kΩ + 2 kΩ，把 5 V 降到 3.3 V

输出方向要注意

MCU 只收不发，所以不存在倒灌



计时为什么要用捕获

外部中断计时

进中断才有时间戳

响应延迟随负载抖动

高电平期间会被插队

输入捕获

边沿一到，硬件把 CNT 锁进 CCR

与中断响应快慢无关

精度 = 1 个计数周期

比喻：输入捕获是“芯片自己按下快门”，CPU 什么时候来取，都不影响照片里的数字

为什么只要一个通道

双通道

CH1 抓上升、CH2 抓下降

逻辑简单、不用状态机

占两个通道、多一根线

单通道

只接 CH1，一轮内换极性

省资源，走线干净

要一个 sm状态位记“在等哪个沿”

核心动作只有两步：抓到上升沿立刻改成下降沿，算完再改回上升沿

定时器怎么配

TIM2 输入捕获关键配置

配置项	取值	为什么
定时器时钟	72 MHz	APB1 预分频≠1 时定时器时钟 ×2
预分频 PSC	71	72 MHz ÷ 72 = 1 MHz，1 计数 = 1 μs
重装载 ARR	0xFFFF	满量程 65535 μs ≈ 65.5 ms
计数模式	向上计数	从 0 累加到 ARR 再归零
捕获极性	初始上升沿	先抓 ECHO 变高那一刻
中断	CC2 + Update	溢出不处理，结果直接错

溢出了怎么办

4 m 对应约 23.5 ms

两万多个计数，16 位够用

但起点不一定从0开始

计数没清零就可能翻过一次

溢出就加一次 65536

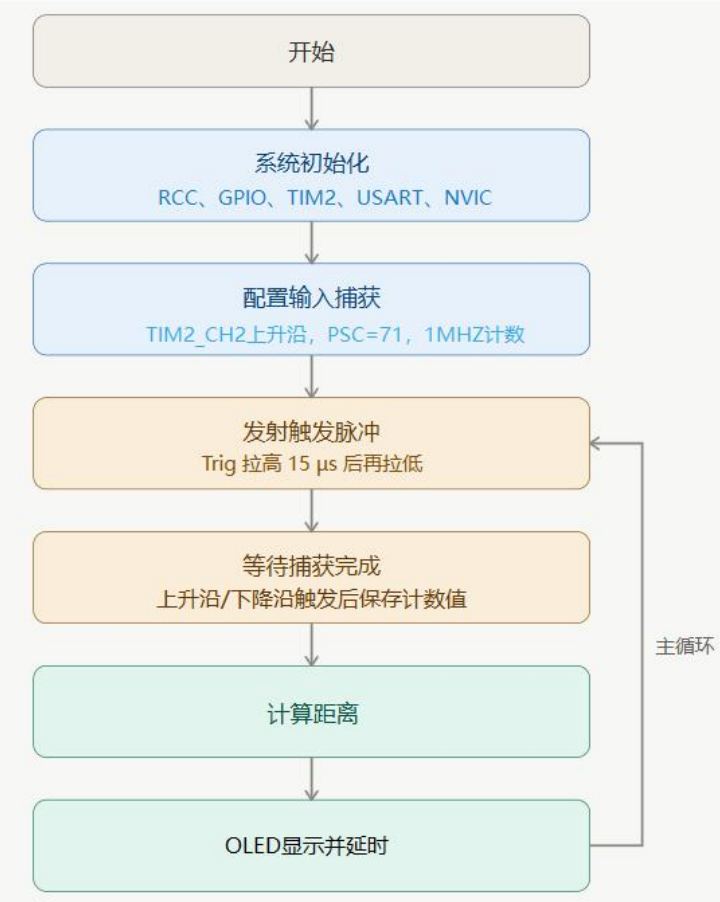
脉宽 = CCR 值 + ovf × 65536

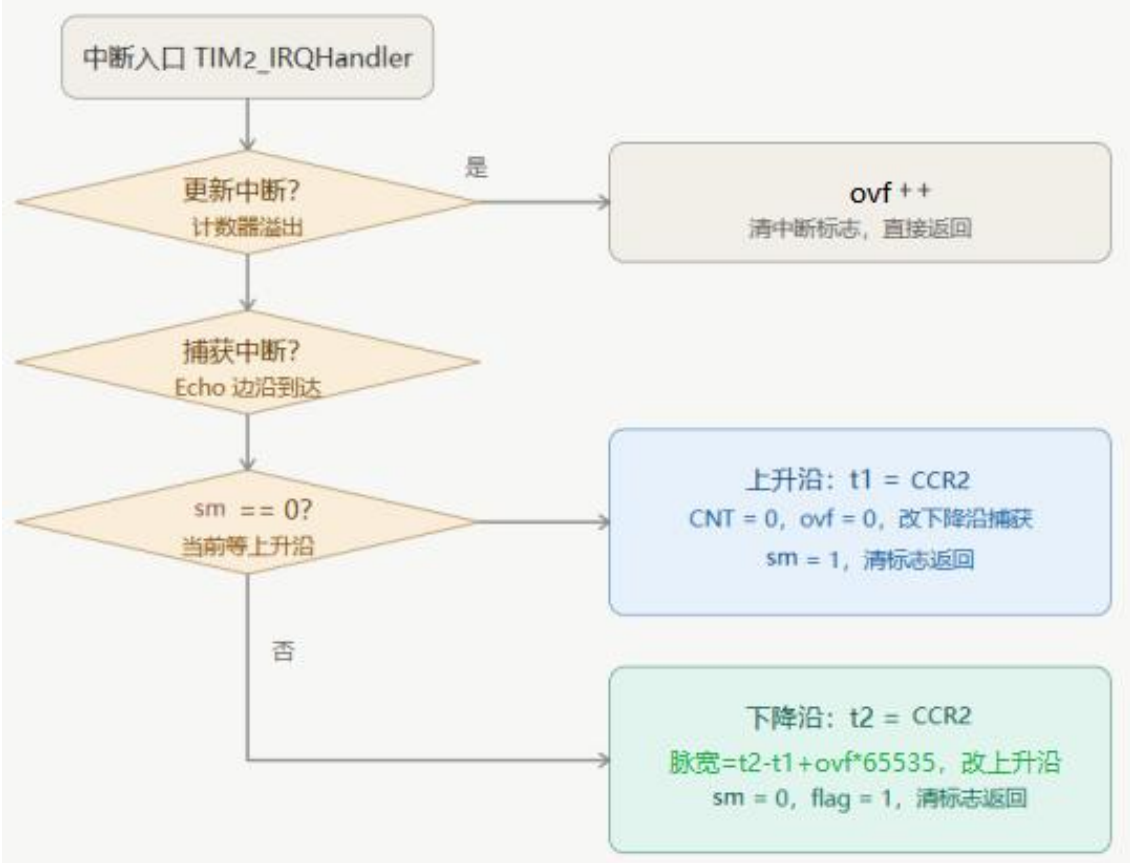
所以必须开 Update 中断

只抓边沿、不管溢出，读数会突然变成 0

> 这一条是单通道捕获唯一容易做错的地方，代码里就是 ovf++ 一行

测距的完整流程





启动信号挂在 MPU6050 的 100 Hz 中断上：每 5 次（50 ms）发一个 15 μs 的高脉冲

误差从哪来

超声测距的五种误差源

误差源	量级	应对
声速随温度变	0.6 m/s 每 ℃	按温度修正 v
2 cm 以内盲区	不可用	装车时留距离
被测面倾斜大于15°	直接丢波	让车头正对障碍
多径与串扰	偶发跳变	间隔 ≥ 60 ms
计时抖动	±1 μs ≈ 0.17 mm	取多次平均

误差 ±3 mm 是理想条件；车上振动、风、软质表面都会让读数跳

程序实现：

```
/* TIM2 CH2 已配好捕获，主循环外先开中断 */
HAL_TIM_IC_Start_IT(&htim2, TIM_CHANNEL_2);

/* 触发：PC15 拉高 15 μs，再拉低 */
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_15, GPIO_PIN_SET);
delay_us(15);                          // ≥10 μs 即可
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_15, GPIO_PIN_RESET);

/* 之后交给回调：Echo 变高记 t1，变低算宽度 */
```

启动挂在 100 Hz 时基上

每 5 次发一发，即 20 Hz

两发之间必须 ≥ 60 ms

否则上一发回波还没散

别在中断里等回波

等 20 ms 会丢一堆别的中断

回调里就是一个状态机

```
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *h)
{
    if (sm == 0) {                          // 上升沿：记起点
        t1 = HAL_TIM_ReadCapturedValue(h, TIM_CHANNEL_2);
        ovf = 0; sm = 1; POL_FALLING;  // 宏：切下降沿
    } else {                                // 下降沿：算宽度
        width = HAL_TIM_ReadCapturedValue(h, TIM_CHANNEL_2)
              + ovf * 65536 - t1;               // 单位 μs
        distance = width * 340 / 20000.0f;     // 单位 m
        sm = 0; flag = 1; POL_RISING;  // 宏：切回上升沿
    }
}
```

这段程序只是简化说明，详细见main.c L354-415

sm 就是“我在等哪个沿”

一个字节解决全部逻辑

宽度单位是微秒

PSC=71 → 1 计数 = 1 μs

算完立刻换回上升沿

下一次测量才接得住

这段是单通道捕获的全部难点：状态机 + 极性切换 + 溢出累加

读数不稳加滤波

```
/* 连续采 5 次，去掉最大最小，剩下取平均 */
uint16_t get_distance(void)
{
    uint16_t v[5], sum = 0, max = 0, min = 0xFFFF;
    for (uint8_t i = 0; i < 5; i++) {
        v[i] = distance_mm();
        if (v[i] > max) max = v[i];
        if (v[i] < min) min = v[i];
        sum += v[i];
    }
    return (sum - max - min) / 3;   // 中值 + 均值
}
```

先剔野值再平均

比单纯平均抗跳变

5 次约 300 ms

对避障足够，对高速不够

返回 0 要判掉

丢波时宽度为 0，别当真

蜂鸣器

无源蜂鸣器

两脚电阻约 15 Ω

内部无振荡电路

要喂 2–4 kHz 方波才响



有源蜂鸣器

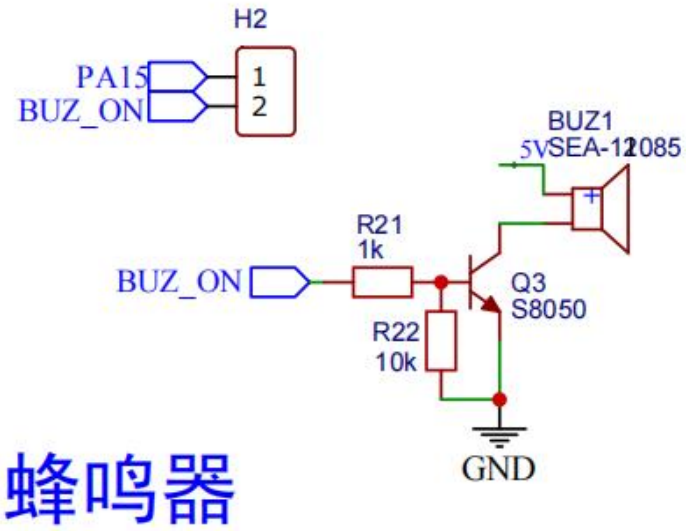
两脚电阻 MΩ 以上

内部自带振荡电路

给电就响，C10B 用的是这种

外观几乎一样，判定量电阻最方便。

C10B硬件接口



蜂鸣器驱动电路

PA15 → 三极管 → 蜂鸣器

IO 只出 3.3 V，电流靠管子放大

高电平驱动

HAL_GPIO_WritePin(GPIOA, ...SET)

跳线帽要插上

H2 不插，板上没有通路

无源蜂鸣器：得喂方波

2 – 4 kHz 方波才响

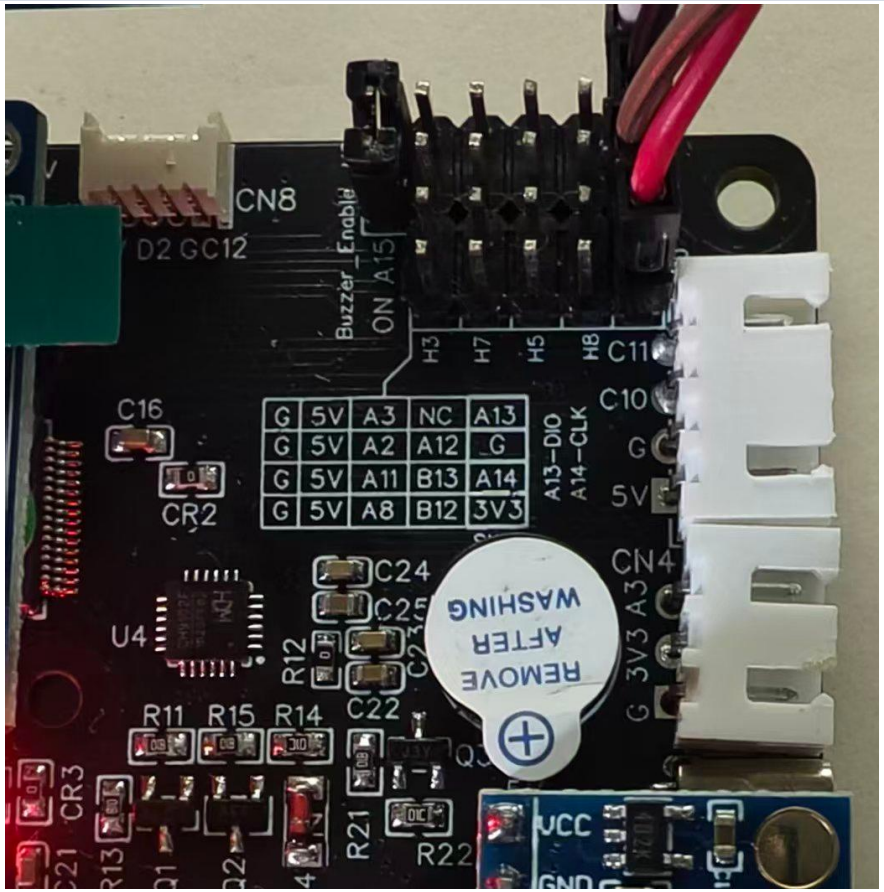
频率决定音调，占空比影响响度

用定时器出 PWM

HAL_TIM_PWM_Start，改 CCR 改频率

别用软件延时翻转

会占满 CPU，还容易跑调



C10B 用的是有源，所以这里只需要 GPIO；换成无源就要多一个定时器

有源蜂鸣器：程序控制很简单

```
GPIO_InitStruct.Pin = beep_Pin;                    //引脚号
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;      //推挽输出
GPIO_InitStruct.Pull = GPIO_NOPULL;                //不上拉、不下拉，浮空输出
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;     //低速模式，功耗更低
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

HAL_GPIO_WritePin(GPIOA, beep_Pin, GPIO_PIN_SET);  //启动蜂鸣器
```

C10B 用的是有源，所以这里只需要 GPIO；

舵机



源自航模运动

控制舵面的伺服电机

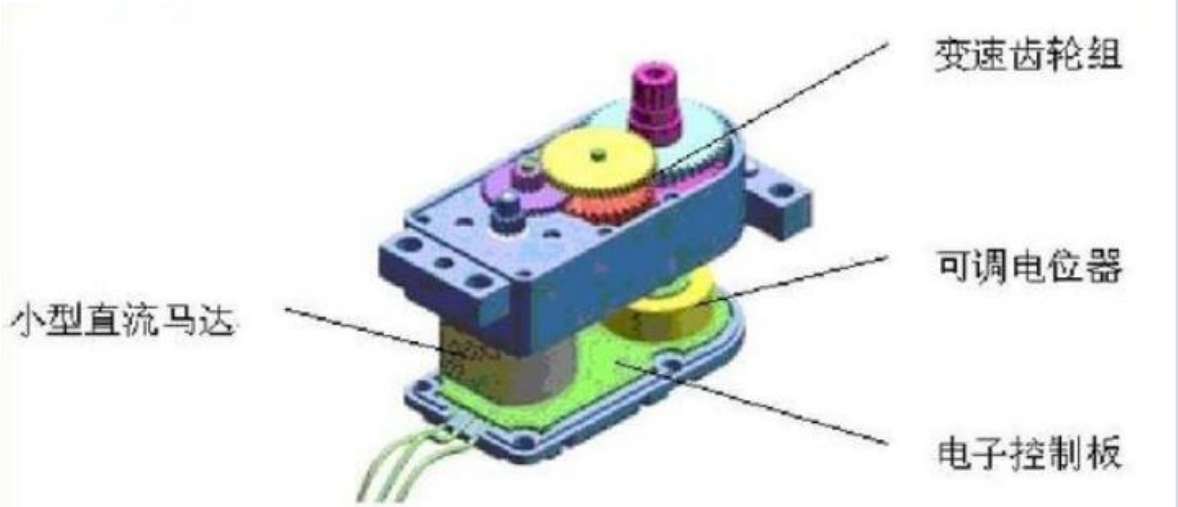
位置（角度）控制

适用于角度不断变化的场合

应用广泛

航模、机器人、机械臂等

舵机结构



舵机内部结构

直流电机 + 齿轮组

堵转扭矩靠减速比堆出来

电位器量角度

转到哪，反馈就是哪

控制板做比较

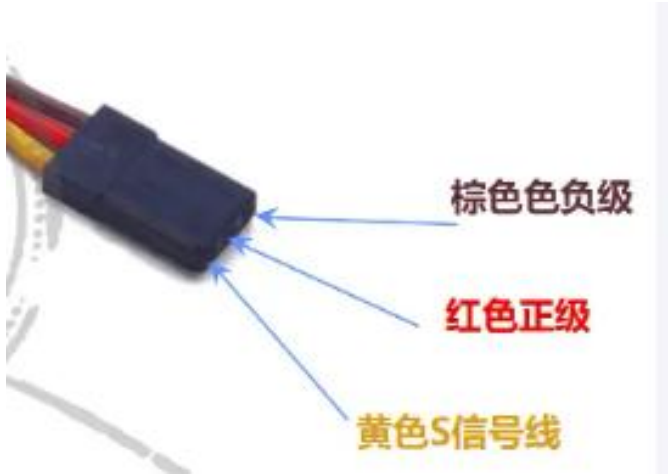
差多少就往回补多少

舵机内部本身就是一个闭环系统

接线：三根线，接反会烧

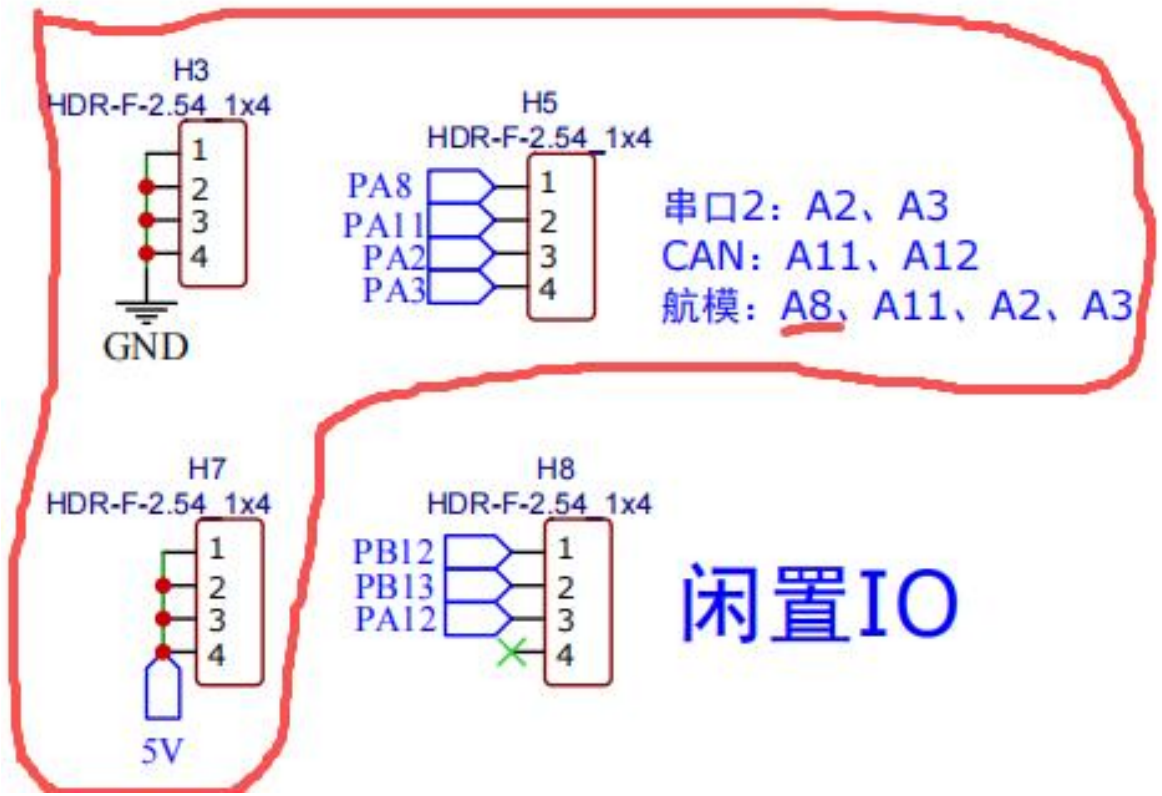
舵机接线的颜色约定

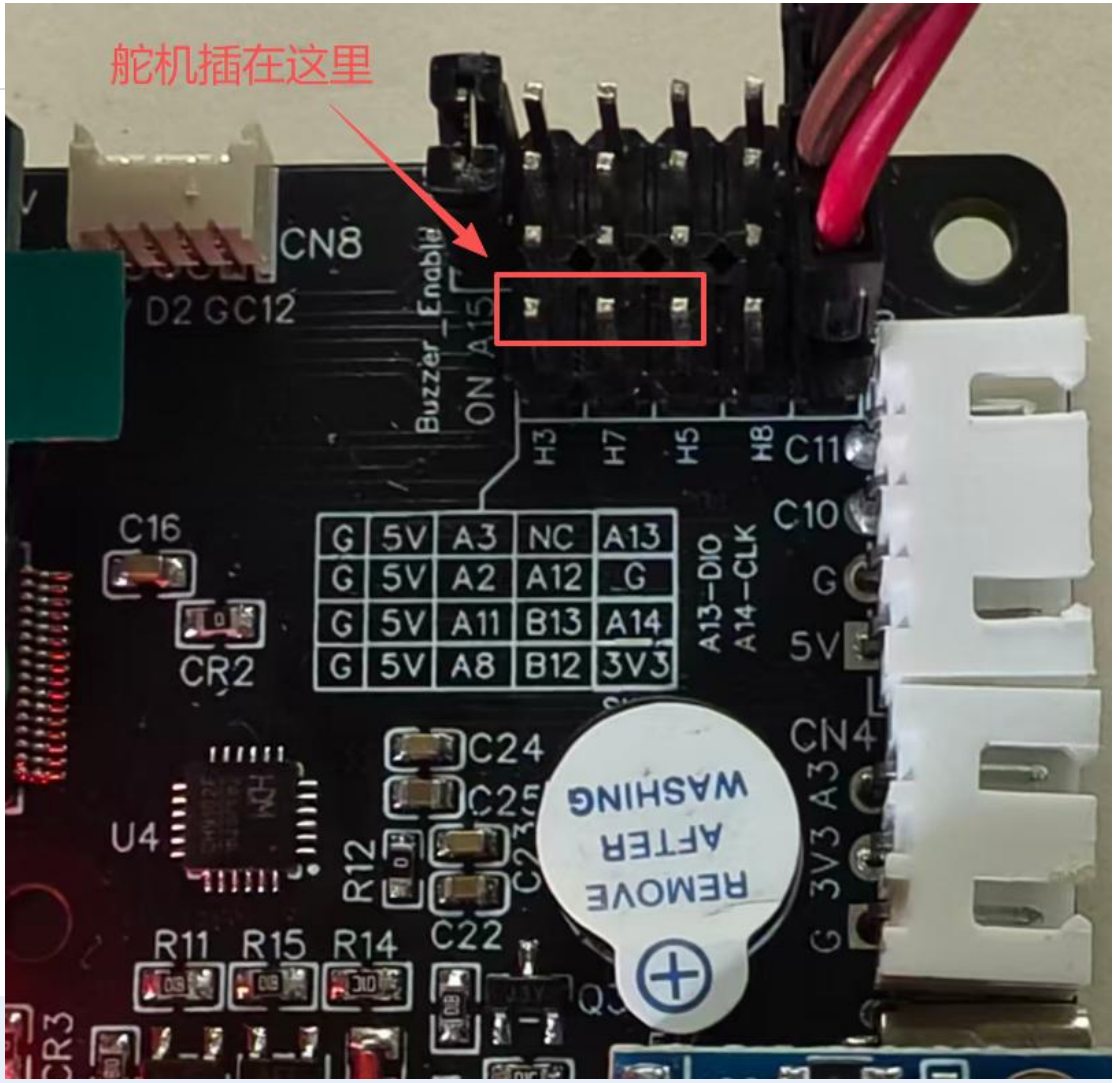
颜色	含义	C10B 接口
棕 / 黑	GND	GND
红	VCC，5 V	5 V（电流可达 1 A）
橙 / 黄	信号，PWM	定时器通道



> 插头没有防呆，仔细检查，再通电

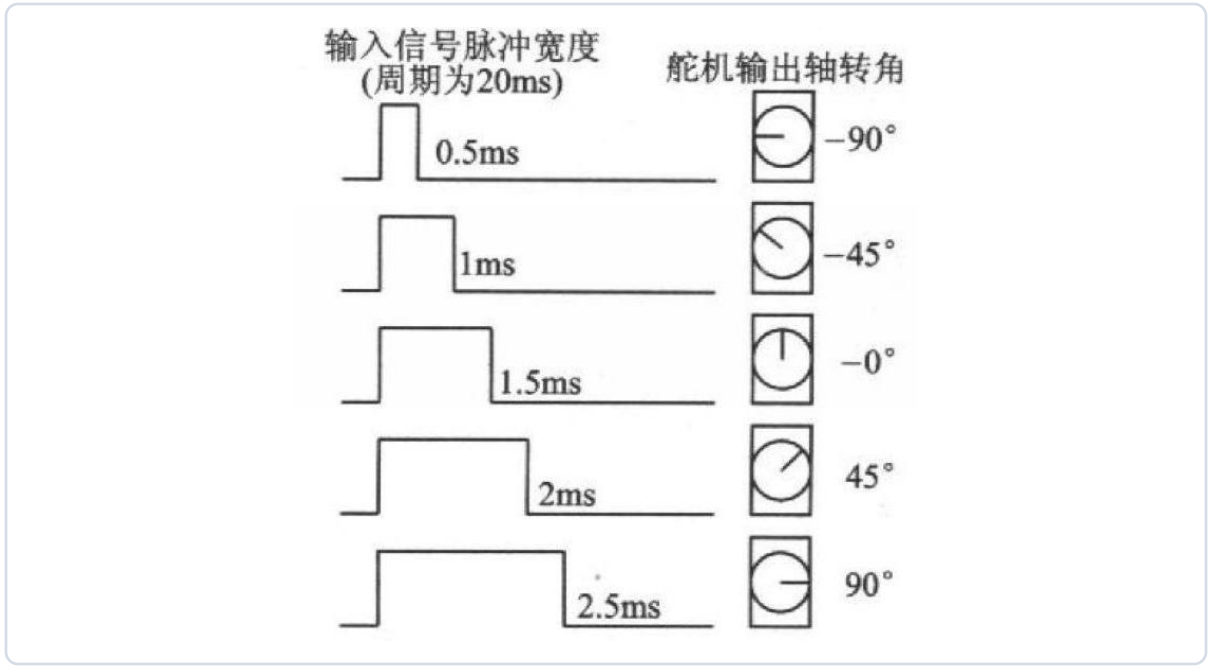
接线：C10B控制板上舵机接口





插头没有防呆，仔细检查，再通电

舵机控制：20 ms 一帧，脉宽定角度



脉宽与转角

周期 20 m

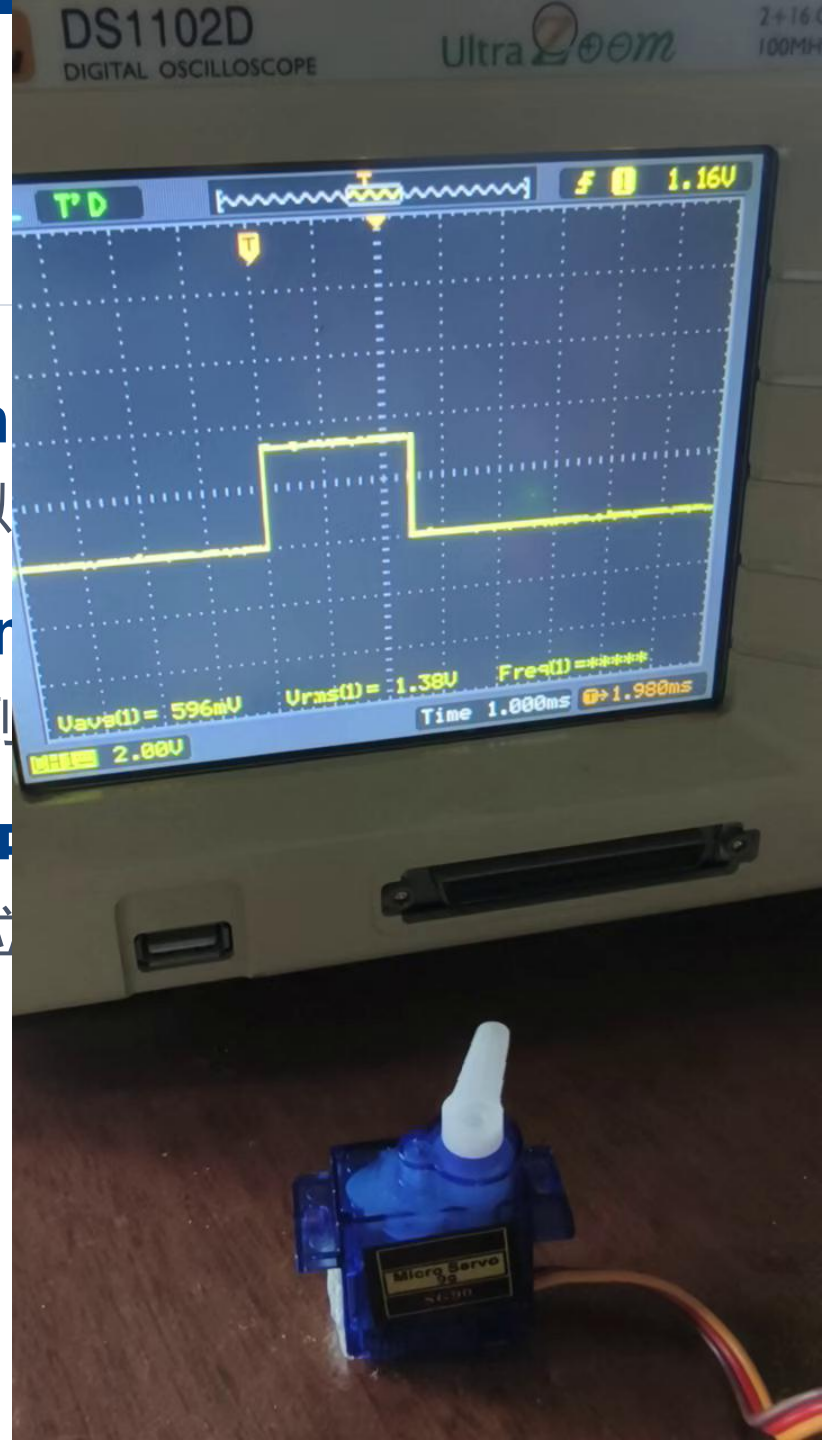
50 Hz，模拟

0.5 – 2.5 r

对应 −90° 到

1.5 ms 是中

上电先给中位



定时器怎么出 50 Hz

PWM 参数怎么算

参数	取值	计算
定时器时钟	72 MHz	APB2
PSC	71	72 MHz ÷ 72 = 1 MHz
ARR	19999	1 MHz ÷ 20000 = 50 Hz
CCR 0°	500	0.5 ms 高电平
CCR 中位	1500	1.5 ms
CCR 90°	2500	2.5 ms

改角度就是改 CCR：1 μs 对应 1 个计数，算起来最省心

程序怎么写才不卡

非阻塞扫角

记一个目标角，每次进主循环走一小步

步进 + 换向

到位就反向，形成往复摆动

按键控制启停

user 键翻转一个 run 标志

用定时器出 PWM

HAL_TIM_PWM_Start，改 CCR 就是改角度

> 判断“卡不卡”的方法很简单：舵机转的时候，OLED 上的数字还在正常跳，就没卡

按键只做一件事：翻标志

按键扫描放主循环

不放在中断，避免抖动干扰时序

消抖 20 ms

检测到变化后延后确认

只翻一个 run 标志

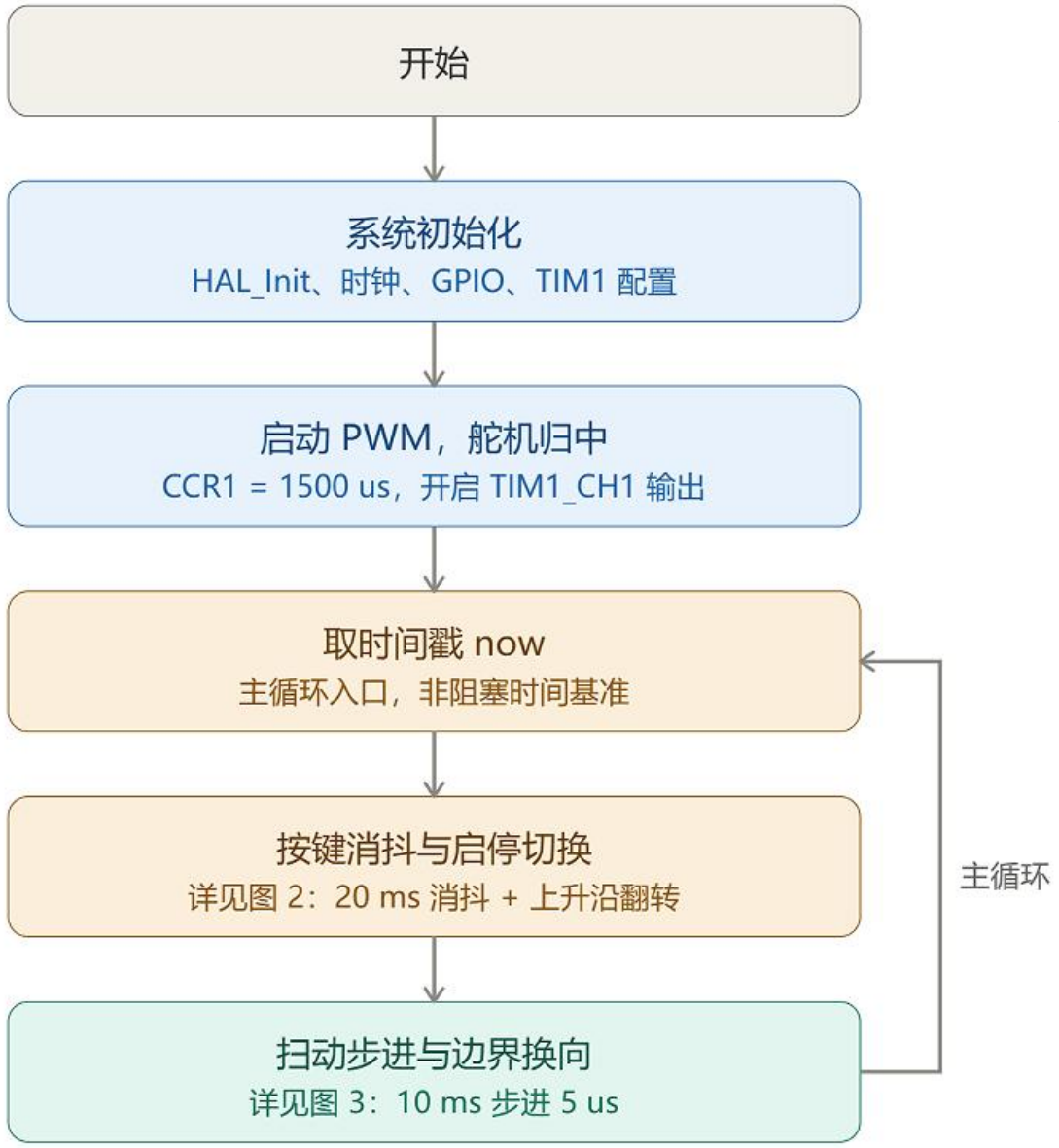
中断里不做长动作

状态与动作分离

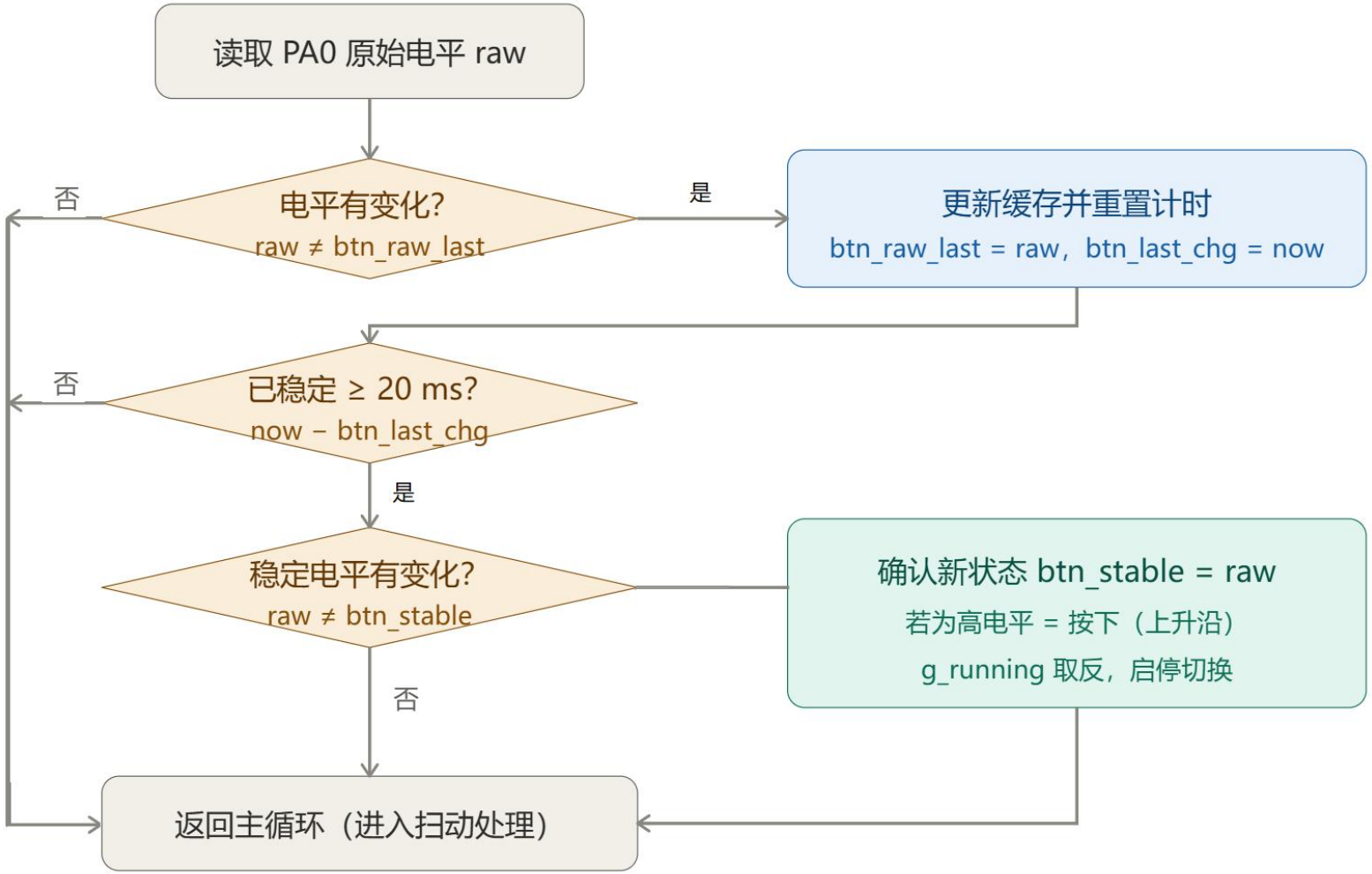
按键管“要不要动”，主循环管“怎么动”

这套写法可以直接搬到任何“按键 + 周期动作”的场景

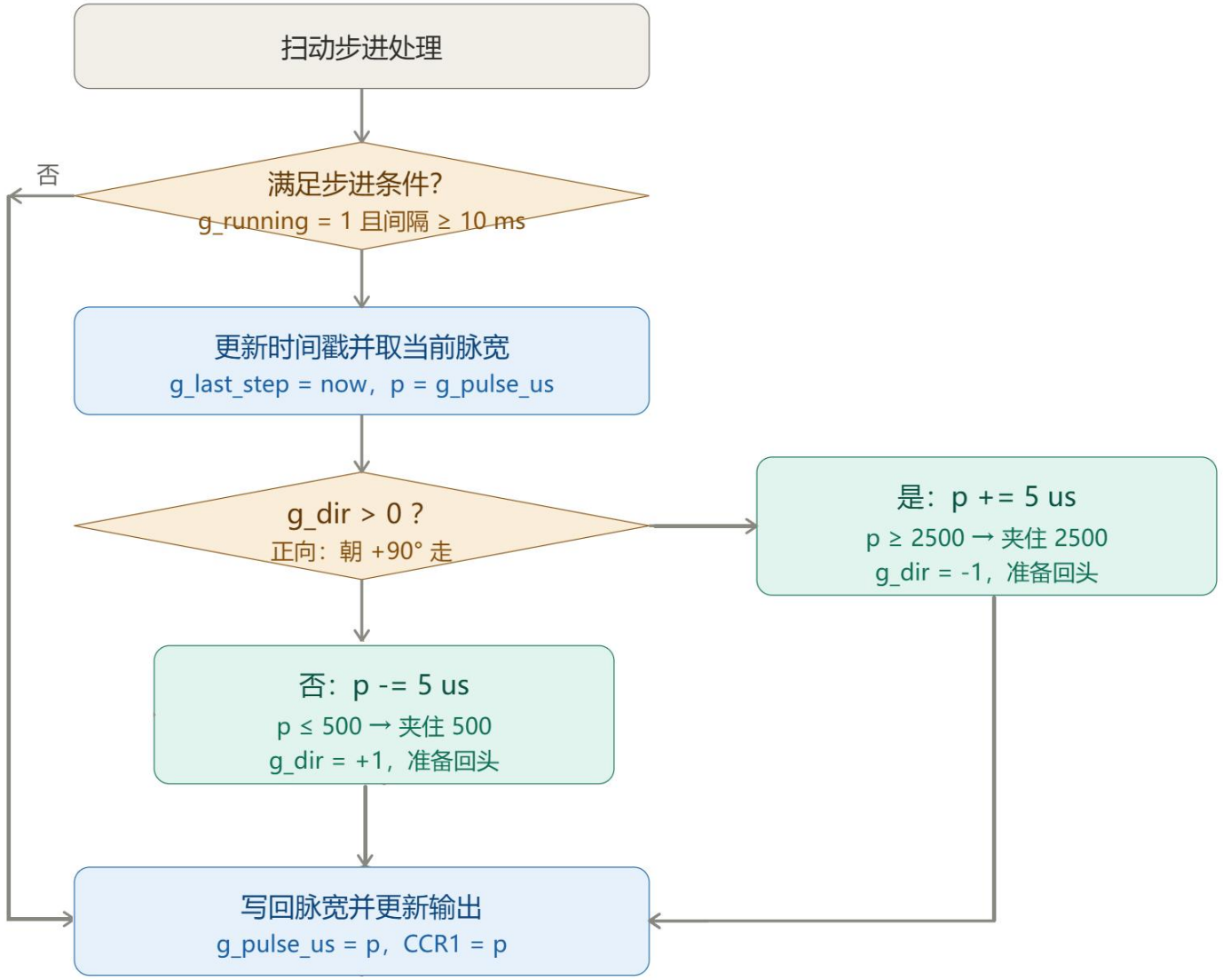
舵机控制流程图


开始
系统初始化
HAL_Init、时钟、GPIO、TIM1 配置
启动 PWM，舵机归中
CCR1 = 1500 us，开启 TIM1_CH1 输出
取时间戳 now
主循环入口，非阻塞时间基准
按键消抖与启停切换
详见图 2：20 ms 消抖 + 上升沿翻转
扫动步进与边界换向
详见图 3：10 ms 步进 5 us
主循环

舵机控制流程图


读取 PA0 原始电平 raw
电平有变化?
raw ≠ btn_raw_last
否
是
更新缓存并重置计时
btn_raw_last = raw，btn_last_chg = now
已稳定 ≥ 20 ms?
now − btn_last_chg
否
是
稳定电平有变化?
raw ≠ btn_stable
确认新状态 btn_stable = raw
若为高电平 = 按下（上升沿）
g_running 取反，启停切换
否
返回主循环（进入扫动处理）

舵机控制流程图


扫动步进处理
满足步进条件?
g_running = 1 且间隔 ≥ 10 ms
否
更新时间戳并取当前脉宽
g_last_step = now，p = g_pulse_us
g_dir > 0 ?
正向：朝 +90° 走
是：p += 5 us
p ≥ 2500 → 夹住 2500
g_dir = -1，准备回头
否：p -= 5 us
p ≤ 500 → 夹住 500
g_dir = +1，准备回头
写回脉宽并更新输出
g_pulse_us = p，CCR1 = p

课后任务：

思考

只使用一个超声波的情况下如何实现更大范围的避障?

CHAPTER 02

02

姿态感知

加速度计看方向，陀螺记过程，DMP 替你算

MPU6050

本讲内容

- 六轴与 MEMS 原理
- 软件 I²C 与初始化
- DMP 固件与四元数
- Yaw 漂移与航向闭环

本讲要get到什么

六轴在测什么

加速度计看方向，陀螺记过程

能看懂软件 I2C

起、止、应答、发字节四个动作

能通过 DMP拿到姿态数据

装 3062 字节固件，读 32 字节一包

知道 Yaw 为什么会漂

如何解决

> 这一讲有点“抽象”，但结论最实用：姿态 = 两个不完美的量互补

MPU6050



MPU-6050 模块

三轴加速度 + 三轴陀螺

±2 g / ±250 °/s起步够用

DMP 硬件解算

不必让 MCU 做姿态融合

AUX I²C 可挂磁力计

这一版没用上

AD0 决定地址

接地 0x68，接高 0x69

六轴到底测什么

三个物理量的量程与用途

测量项	量程	单位	主要用途
三轴加速度	±2 g~±16 g	g	倾角、振动、敲击
三轴角速度	±250 °/s~±2000 °/s	°/s	姿态积分、转动检测
温度	−40 ~ 85	℃	零偏补偿的输入

注意：它不测“角度”，角度是算出来的——这是初学者最大的误会

内部结构

MPU6050 芯片内部

3 轴加速度计（MEMS）
量程 ±2 / ±4 / ±8 / ±16 g，16 位 ADC

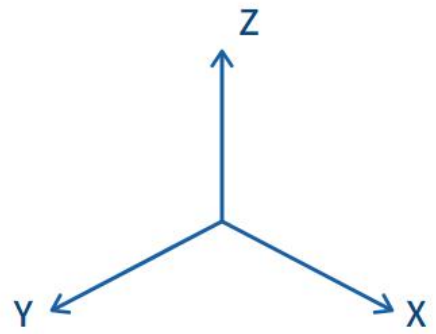
3 轴陀螺仪（MEMS）
量程 ±250 / ±500 / ±1000 / ±2000 °/s

温度传感器
T = 原始值 ÷ 340 + 36.53 °C

DMP 数字运动处理器
硬件解算四元数，本讲义在 MCU 端解算

I2C 从机接口
地址 0x68（AD0 接地）/ 0x69

机体坐标系与姿态角

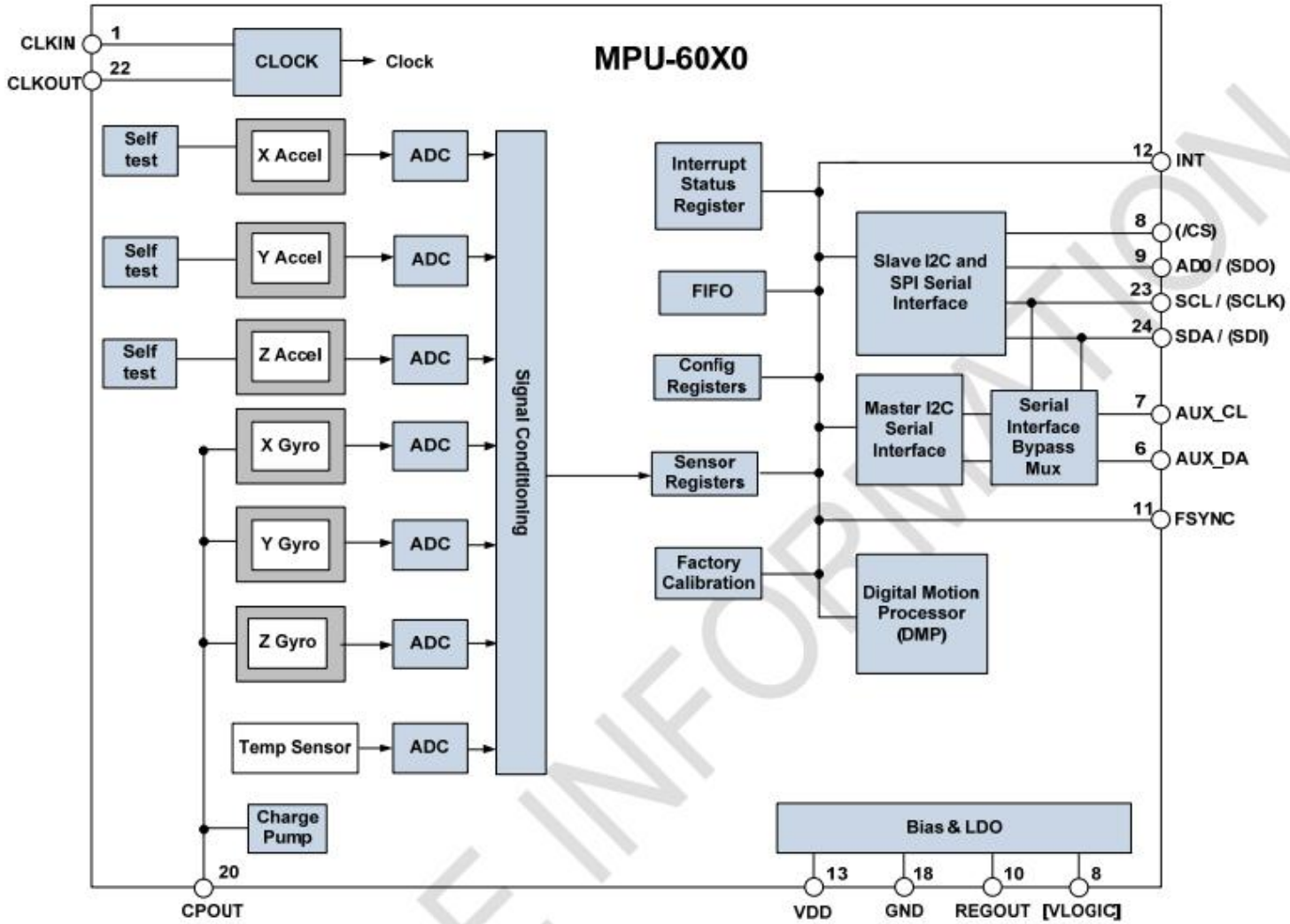


Roll 横滚：绕 X 轴

Pitch 俯仰：绕 Y 轴

Yaw 航向：绕 Z 轴（会漂移）

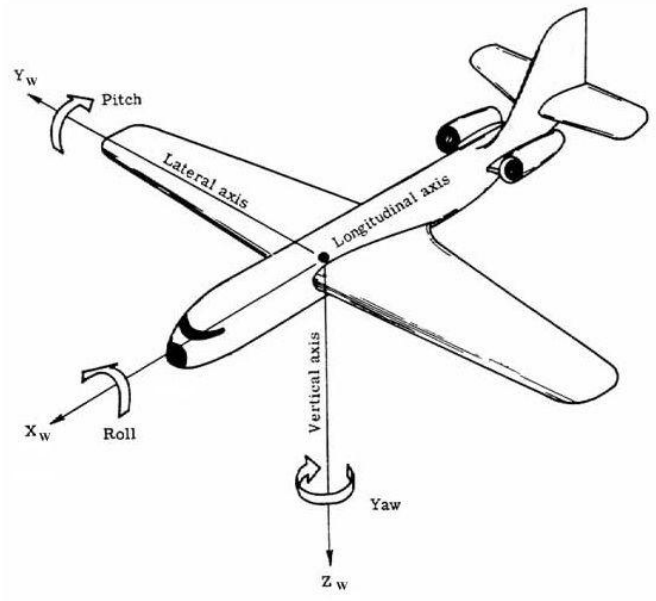
内部框图



Note: Pin names in round brackets ( ) apply only to MPU-6000
Pin names in square brackets [ ] apply only to MPU-6050

NCE INFORMATION

三个姿态角



姿态角示意

Roll 横滚

绕 X 轴，机头方向那条线

Pitch 俯仰

绕 Y 轴，抬头低头

Yaw 航向

绕 Z 轴，左右转头

记忆法：Yaw 是“回头看”，Pitch 是“点头”，Roll 是“歪头”

MPU6050与ICM20948

MPU6050 与 ICM20948 的区别

对比项	MPU6050（C10B）	ICM20948（C50C）
轴数	6 轴：加速度 + 陀螺	9 轴：再加地磁
航向 Yaw	长期漂移，需外挂磁力计	内置地磁，无漂移
供电 IO	3.3 / 5 V 兼容	1.8 V 内核，要电平转换
通信	仅 I²C	I²C + 7 MHz SPI
DMP 能力	仅 6 轴融合	硬件 9 轴融合
开发难度	极低，资料丰富	偏高，寄存器复杂

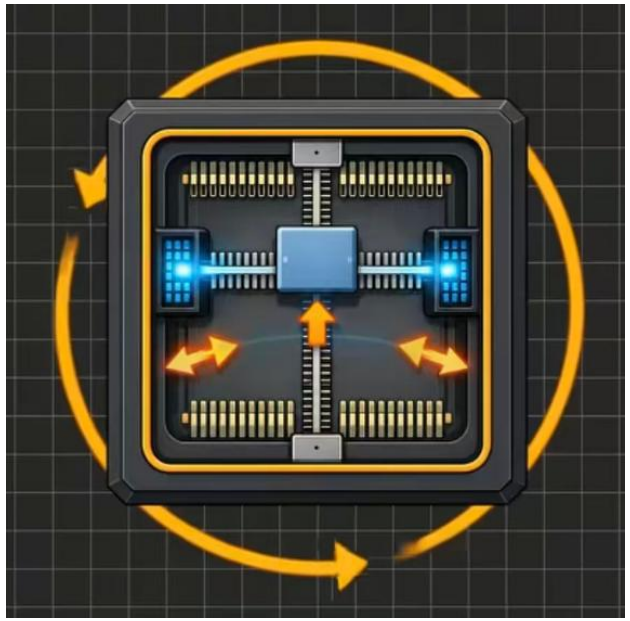
选型不是越新越好：要根据需求和实际选，MPU6050 刚好

InvenSense 三代怎么分

三代产品的核心差异

型号	代次	核心特征
MPU6050	一代 6 轴	仅 I²C，无地磁，DMP 只融合 6 轴
MPU9250	二代 9 轴	加 AK8963 地磁，地磁需软件融合
ICM20948	三代 9 轴	内置 AK09916，硬件原生 9 轴 DMP

三轴加速度计：质量块+差分电容



MEMS 质量块结构

加速度让质量块位移

改变与固定极板的电容

测电容差即得加速度

静止时 Z 轴读数约 1 g

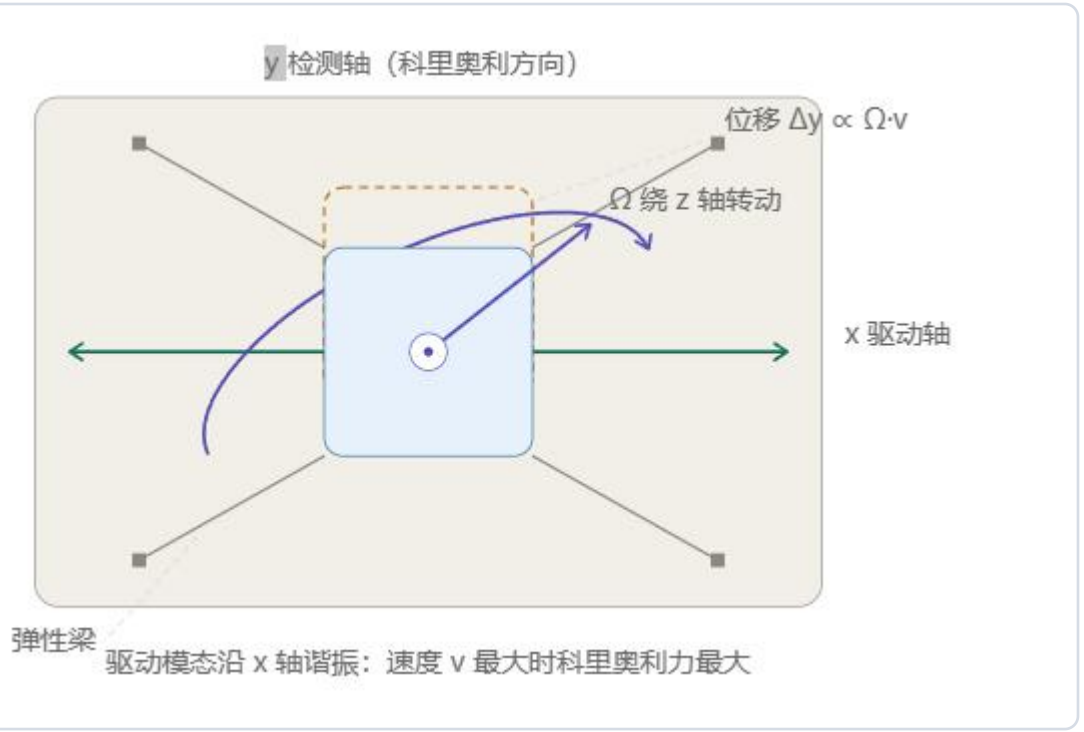
所以能算 Roll / Pitch

重力投影比例就是倾角

但怕振动

运动加速度混进来，角度立刻抖

陀螺仪：科里奥利力



振动让质量块产生位移

检测位移即得角速度

角速度积分才能得到角度

积分会累积零偏误差

所以短期准、长期飘

静止时三个轴各读多少

重力在三个轴上的投影

姿态	ax	ay	az
水平放置	0	0	1 g
头低 30°	0.5 g	0	0.87 g
侧倾 90°	0	1 g	0
倒扣	0	0	−1 g

> az 从 1 g 掉到 0.87 g 就是倾了 30°：cos30° ≈ 0.866

两种 MEMS，互补才实用

加速度计

量重力投影，看“方向”

长期准，不随时间漂

短期抖，一动就乱

陀螺仪

量角速度，记“过程”

短期准，长期漂

要积分，误差越积越大

一个长期准、一个短期准，合起来才可用——这就是互补滤波的全部动机

八个关键寄存器

最常用的寄存器（最好配合手册看）

地址	寄存器	作用与推荐值
0x75	WHO_AM_I	返回 0x68，第一件事就是验通信
0x6B	PWR_MGMT_1	0x80 复位，0x00 唤醒
0x1B	GYRO_CONFIG	FS_SEL：0x18 → ±2000 °/s
0x1C	ACCEL_CONFIG	AFS_SEL：0x00 → ±2 g
0x19	SMPLRT_DIV	输出频率 = 1 kHz ÷ (DIV+1)
0x1A	CONFIG	DLPF_CFG，带宽取采样率一半
0x3B	ACCEL_XOUT_H	连续 14 字节：加 6 + 温 2 + 陀 6

改单个位，不能整字节覆盖

```
static void IICwriteBits(uint8_t reg, uint8_t bitStart,
                         uint8_t length, uint8_t data)
{
    uint8_t mask = (0xFF << (bitStart + 1))
                 | (0xFF >> (8 - bitStart + length - 1));
    data <<= (8 - length);
    data >>= (7 - bitStart);
    bsp_IICread(dev, reg, &v, 1);   // 先读回原值
    v = (v & mask) | data;          // 只改目标位
    bsp_IICwrite(dev, reg, &v, 1);  // 再写回
}
```

先读再写

否则会把同寄存器其他位清掉

掩码把目标位挖空

其余位保持 1，与运算后不变

DMP 初始化大量用它

所以这段读不懂就看不下去

一次突发读 14 个字节

0x3B 起连续 14 字节

加速度 6 + 温度 2 + 陀螺 6

每轴都是高字节在前

拼装时别把高低位弄反

一次读完比分六次读快

也避免读到两帧拼在一起

温度换算

T = 36.53 + regval / 340

温度不是装饰：陀螺零偏随温度变，做温补时这 2 个字节就是输入

芯片内置温度作用

换算公式

T = 36.53 + regval / 340

用途一：温补

陀螺零偏随温度走，要按温度查表

用途二：看是否过热

持续 60 ℃ 以上要怀疑供电

用途三：验证读数

室温 25 ℃ 左右，明显偏离说明读错字节

> 顺手一读，代价是两个字节；做温补时它就是唯一的输入

硬件接口

MPU6050 与 C10B 的接线

模块引脚	STM32F103	说明
VCC	5 V	模块供电
GND	GND	必须共地
SCL	PB14	软件模拟时钟，建议开漏
SDA	PB15	软件模拟数据，建议开漏
AD0	4.7 kΩ 接地	地址 0x68；接 3.3 V 为 0x69
INT	PB9	下降沿触发，100 Hz 时基

F103 这组引脚（PB14/PB15）没有硬件 I²C，所以只能软件模拟

模块安装有讲究

装在车体重心附近

平移加速度对它的影响最小

刚性固定，别用双面胶

软连接会让振动变成噪声

远离电机与电源线

PWM 线的磁场直接进陀螺

加减震垫

几十赫兹的振动压下去，读数立刻干净

> “数据乱跳”十有八九是机械问题：先拧紧螺丝，再改代码

开漏还是推挽

推挽输出

能主动输出高和低

两头发不同电平时硬碰硬

大电流，可能烧引脚

开漏输出

只能拉低，“1”靠上拉电阻

天然实现线与（I²C 的根基）

模块自带 4.7 kΩ 上拉

工程里为什么采用了推挽输出？

I²C 是多主多从的“线与”总线：谁都可能拉低，谁都不拉时才被上拉电阻拉高

地址里藏着读写位

7 位地址 0x68

AD0 接地；接 3.3 V 则是 0x69

写操作 0xD0

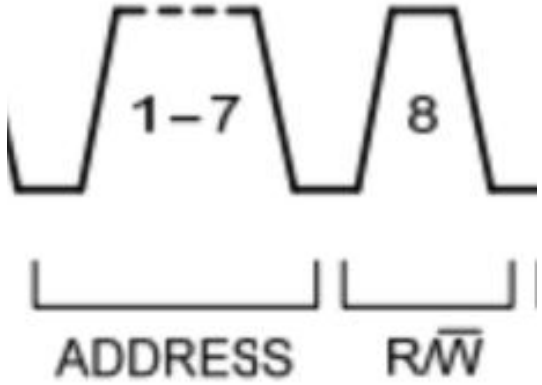
0x68 << 1 | 0

读操作 0xD1

0x68 << 1 | 1

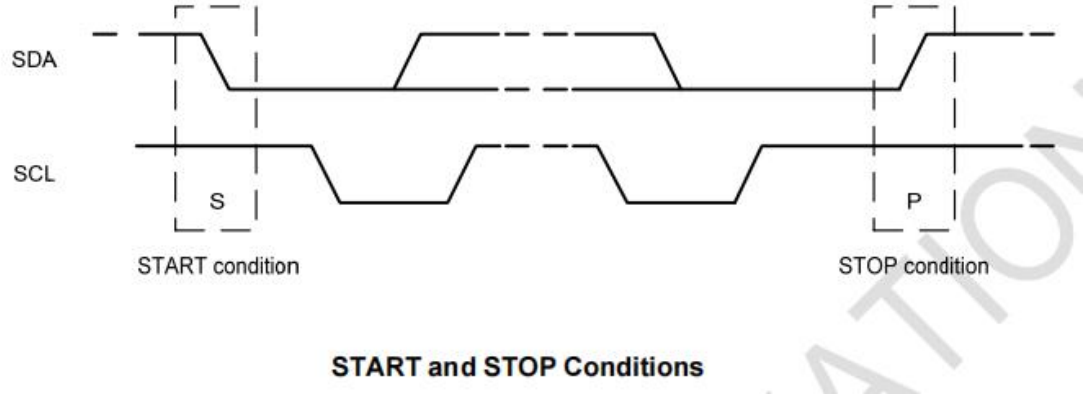
手册里写法不一

有的给 7 位，有的给 8 位，差一位



> 读不到 0x68 时先确认对方用的是 7 位还是 8 位写法，再查接线

四个动作撑起整条I2C总线



起始与停止信号

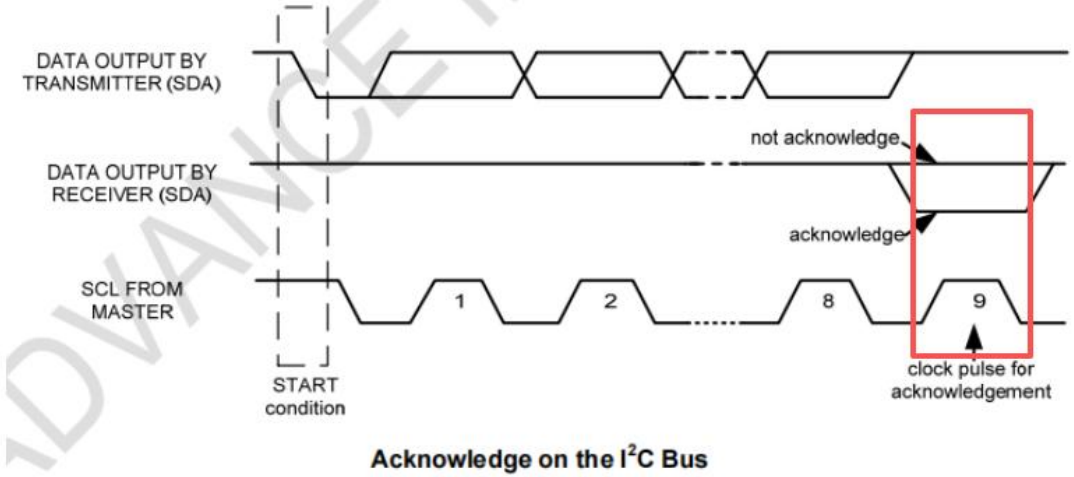
起始 S

SCL 高时，SDA 由高变低

停止 P

SCL 高时，SDA 由低变高

四个动作撑起整条I2C总线

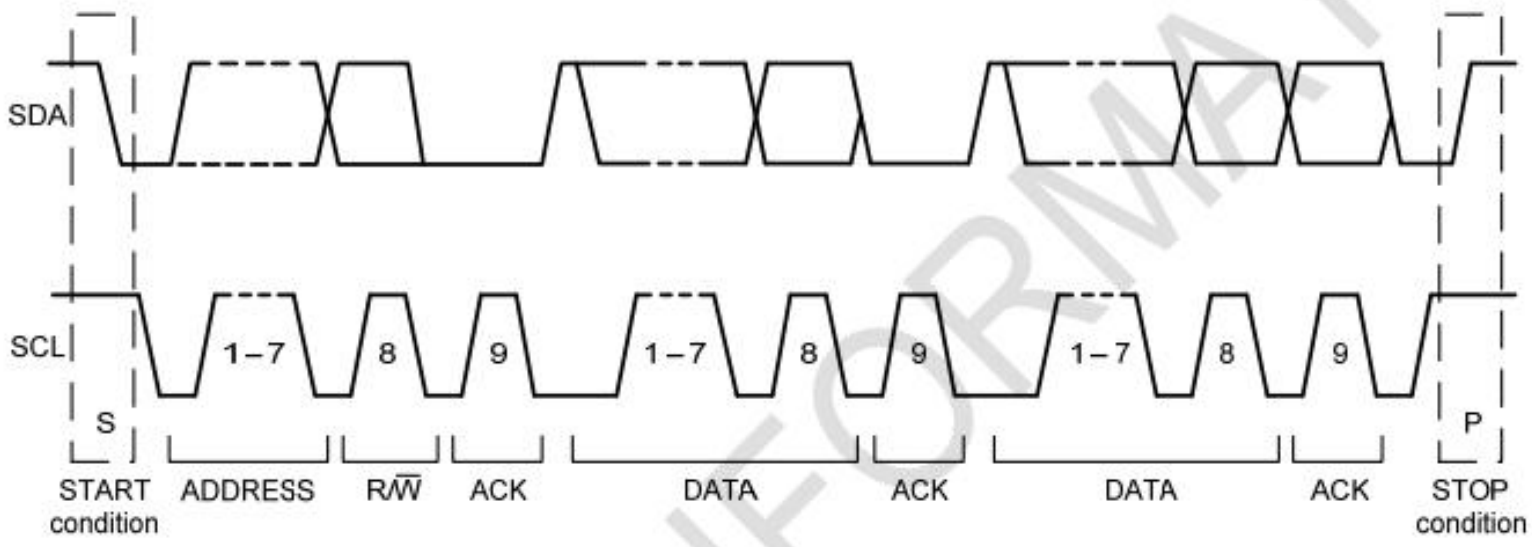


应答信号

应答 ACK

第 9 个时钟，从机把 SDA 拉低

四个动作撑起整条I2C总线



Complete I²C Data Transfer

数据发送信号

一张函数指针表，换总线只改这里

```
typedef struct {
    IIC_Status_t (*read_reg)(uint16_t dev, uint16_t mem,
                             uint8_t *buf, uint16_t size);
    IIC_Status_t (*write_reg)(uint16_t dev, uint16_t mem,
                              uint8_t *buf, uint16_t size);
    void (*delay_ms)(uint16_t ms);
} IICInterface_t;

/* 软件 I2C 的实例；换硬件 I2C 只换这一张表 */
IICInterface_t User_sIICDev = {
    .read_reg  = IIC_Mem_Read,
    .write_reg = IIC_Mem_Write };
```

上层只认接口

MPU6050.c 不关心底层是软是硬

.init 是空函数

引脚已在 MX_GPIO_Init 里配好

delay_ms 也进了表

换平台时延时要一起换

起始信号就三行

```
static void IIC_Start(void)
{
    SDA_OUT();
    IIC_SCL = 0;
    IIC_SDA = 1;
    IIC_SCL = 1;
    delay_us(1);
    IIC_SDA = 0;                // SCL 高时拉低 = 起始
    delay_us(1);
    IIC_SCL = 0;
    delay_us(1);
}
```

// 停止与其对称：SDA 由低变高，在 SCL 高电平时完成

延时不能省

太快从机跟不上，数据读不到

停止后要留总线空闲

读写都要切方向

SDA_OUT / SDA_IN 别忘了

读寄存器要发两次起始

```
sIIC_Start();
sIIC_SendByte(dev_addr);           // 写
sIIC_WaitAck(timeout);
sIIC_SendByte(mem_addr);           // 寄存器号
sIIC_WaitAck(timeout);
sIIC_Start();                      // 重复起始 Sr（不插 Stop）
sIIC_SendByte(dev_addr | 0x01);    // 读
sIIC_WaitAck(timeout);
for (i = 0; i < size; i++)
    data[i] = sIIC_ReadByte(i == size - 1 ? 0 : 1);
sIIC_Stop();
```

为什么中间不能 Stop

Stop 会释放总线，其他主机可能插进来

最后一个字节回 NACK

告诉从机“别再给了”

每次 ACK 都要判超时

否则总线卡死，整机像死机

初始化MPU6050

01 **初始化 I²C 引脚**
PB14 / PB15 推挽输出，PB9 上拉输入

02 **复位器件**
0x6B 写 0x80，等 100 ms，再写0x00唤醒

03 **选时钟源**
0x6B 写 0x01，用 X 轴陀螺 PLL，时钟精度更高

04 **设量程并唤醒**
0x1B = 0x18，0x1C = 0x00，使能0x6C = 0x00

> 复位后 0x6B 默认为 0x40（睡眠位为 1），不写 0x00 就永远醒不过来

时钟源为什么不用内部 RC

CLKSEL[2:0] 时钟源选择

CLKSEL	时钟源	评价
000	内部 8 MHz RC	默认，精度差，会拖累采样
001	X 轴陀螺 PLL	最常用，本工程选这个
010	Y 轴陀螺 PLL	同上，看走线方便
011	Z 轴陀螺 PLL	同上
100	外部 32.768 kHz	低功耗场景
111	关闭时钟	时序电路复位，不用于正常工作

量程与灵敏度，一对一的数

FS_SEL / AFS_SEL 与灵敏度

档位	陀螺量程	灵敏度	加速度量程	灵敏度
0	±250 °/s	131 LSB/(°/s)	±2 g	16384 LSB/g
1	±500 °/s	65.5	±4 g	8192
2	±1000 °/s	32.8	±8 g	4096
3	±2000 °/s	16.4	±16 g	2048

代码里的 16384 和 131 就是这张表查出来的，改量程必须同时改除数

采样率、DLPF 与 FIFO

采样率 = 1 kHz ÷ (DIV+1)

DIV = 9 得 100 Hz，DMP 也按这个节拍出包

DLPF 带宽取采样率一半

100 Hz 采样就配约 42 Hz 带宽

FIFO 打开对应通道

0x23 一个位管三个轴

顺序有讲究

先设 DLPF，再算分频，最后开 FIFO

> DLPF 是抗振的第一道闸：车体振动多在百赫兹以上，压到 42 Hz 立刻就干净了

FIFO 里的一包，32 字节

DMP 输出包的结构

内容	字节数	说明
四元数 q30	16	四个 int32，放大 2^{30} 倍
陀螺原始值	6	三轴，各 2 字节，高位在前
加速度原始值	6	三轴，各 2 字节
手势 / 其他	4	本工程未使用

包长必须和 dmp_enable_feature 打开的功能一致，否则解析全错位

DMP 是什么

芯片里的一个小协处理器

专门做 6 轴姿态融合

固件要自己装进去

3062 字节，分 192 包写入约 190 ms

算完写进 FIFO

每 10 ms 一个包，32 字节

MCU 只做两件事

取包、把四元数换成欧拉角

> 没有 DMP，你就得在 MCU 里跑卡尔曼或互补滤波，还得自己调参

3062 字节怎么灌进去

按块写，不是一次写完

mpu_write_mem 每次 16 字节，共 192 包

写完回读校验

读回来比一遍，不一致就失败

整个加载约 190 ms

所以上电初始化要留足时间

失败就重来

I²C 干扰大时偶发失败，重试一次通常就好

> 这一步最容易被“上电太快”坑：固件还没装完就开始读，永远是 0

DMP初始化

MPU6050初始化后读出6轴原始数据

原始数据对小车用处不大

期望得到的是姿态数据

欧拉角：航向角（yaw）、横滚角（roll）和俯仰角（pitch）

DMP专门做姿态解算

大大降低MCU的负担

> 读源码从下往上：先会看 I²C，再看 inv_mpu，最后才是 main 里那两行

DMP库

DMP 驱动库的文件分工

文件	行数	职责
bsp_siic.c	板级	软件 I²C 的物理时序
MPU6050.c/.h	429	二次封装：DMP_Init、Read_DMP
inv_mpu.c/.h	2885	寄存器读写、FIFO、DMP 使能
inv_mpu_dmp_motion_driver.c/.h	1367	3062 B 固件 + 功能配置
dmpKey.h / dmpmap.h	494 / 264	逻辑键 → 物理地址

读源码从下往上：先会看 I²C，再看 inv_mpu，最后才是 main 里那两行

DMP 初始化流程



关键三处：装固件、设 FIFO 速率、最后 mpu_set_dmp_state(1) 才让 INT 开始跳

取包到出角度，就这一个函数

```
uint8_t Read_DMP(void)
{
    dmp_read_fifo(gyro, accel, quat, &ts, &sensors, &more);
    if (!(sensors & INV_WXYZ_QUAT)) return 1;   // 没取到
    q0 = quat[0]/q30; q1 = quat[1]/q30;
    q2 = quat[2]/q30; q3 = quat[3]/q30;
    Pitch = asin(-2*q1*q3 + 2*q0*q2) * 57.3f;
    Roll  = atan2(2*q2*q3 + 2*q0*q1,
                  1 - 2*q1*q1 - 2*q2*q2) * 57.3f;
    Yaw   = atan2(2*(q1*q2 + q0*q3),
                  q0*q0+q1*q1-q2*q2-q3*q3) * 57.3f;
    return 0;
}
```

返回值要判

取失败时 quat 还是上一包的值

q30 是常量 2^{30}

不除会得到天文数字

全局变量给主循环用

OLED 与上报都读这三个值

姿态解算方法选择

四种姿态解算路线

解法	优点	代价
只用加速度	没有积分漂移	车一动就抖
只用陀螺	平滑、跟手	误差越积越大
互补滤波	便宜好用，α = 0.98	参数得自己调
DMP	硬件算，几乎不占 CPU	只给四元数，看不到中间量

互补滤波α 怎么调

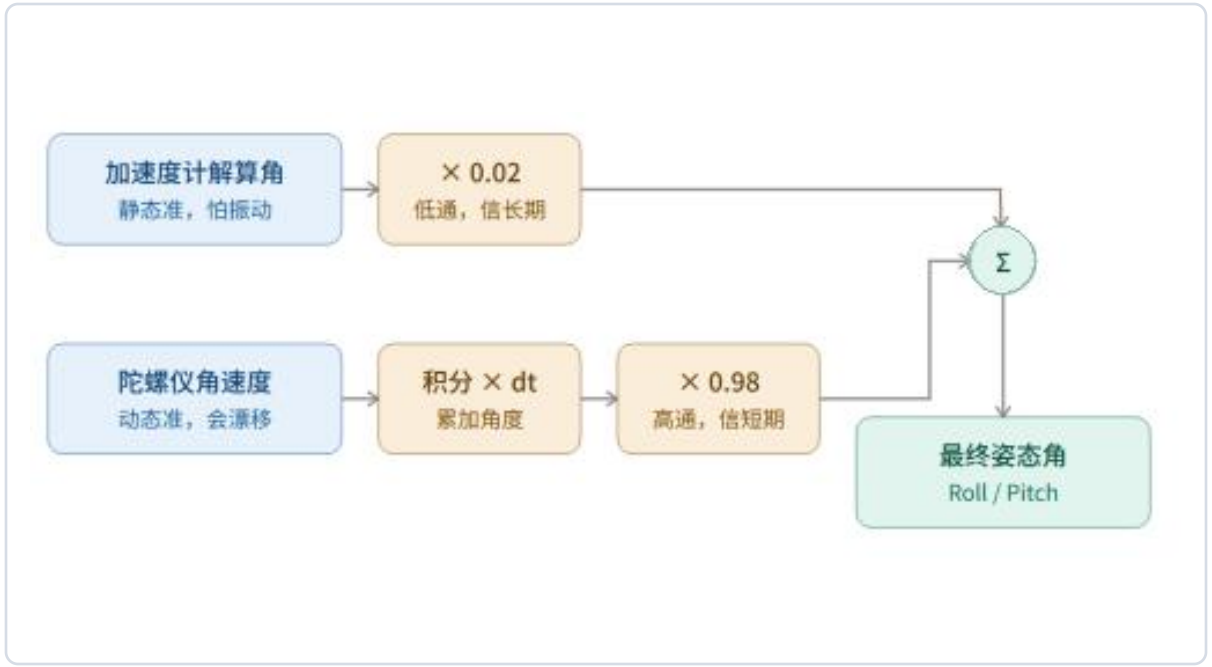
互补滤波的 α 选择

现象	α 往哪调	说明
角度跟着振动抖	调大（更信陀螺）	0.98 → 0.99
角度慢慢偏走	调小（更信加速度）	0.98 → 0.97
起步时角度追不上	调小	加速度计拉回更快
整体都还行	别动	0.98 是多数场景的默认

```
/* 3. 互补滤波: 陀螺积分为主, 加速度慢慢把它拉回来 */
g_roll = ALPHA * (g_roll + gx * dt) + (1.0f - ALPHA) * acc_roll;
g_pitch = ALPHA * (g_pitch + gy * dt) + (1.0f - ALPHA) * acc_pitch;
g_yaw += gz * dt; /* Yaw 无绝对参考, 只能积分 */
```

> 调参有依据：抖 = 高频问题（压低通），漂 = 低频问题（压高通）

互补滤波



互补滤波框图

低通 + 高通

陀螺走高通，加速度走低通

α 决定信谁

0.98 表示 98% 信陀螺

振动大就降 α

0.99 / 0.01；漂得凶就 0.97 / 0.03

滤波本体只有五行

```
#define ALPHA 0.98f
ax = raw->ax / 16384.0f;
gx = raw->gx / 131.0f - gxo;    // 减零偏!

acc_roll = atan2f(ay, az) * 57.3f;
g_roll  = ALPHA * (g_roll + gx * dt)
        + (1 - ALPHA) * acc_roll;
g_yaw  += gz * dt;    // 无绝对参考
```

先换算再滤波

原始值除以灵敏度才是物理量

零偏必须减

不减的话积分几秒就歪

Yaw 只能纯积分

这就是它必漂的根本原因

零偏校准：静止采 200 次

```
void MPU6050_Calibrate(float *gxo, float *gyo, float *gzo)
{
    int32_t sx = 0, sy = 0, sz = 0;  MPU_Raw_t r;
    for (int i = 0; i < 200; i++) {
        MPU6050_ReadRaw(&r);
        sx += r.gx; sy += r.gy; sz += r.gz;
        Delay_ms(5);
    }
    *gxo = (float)sx / 200.0f / 131.0f;
    *gyo = (float)sy / 200.0f / 131.0f;
    *gzo = (float)sz / 200.0f / 131.0f;
}
```

1 秒静止，换来 10 倍提升

漂移从每分钟 10° 压到 1° 量级

必须在车上静止做

手拿着抖，平均出来的零偏是错的

做完再开始积分

顺序错了，前几秒就把误差积进去

上电自检与校准的顺序

01 **读 WHO_AM_I**
不是 0x68 就别往下走，先查线

02 **软复位 + 唤醒**
0x6B 写 0x80，等 100 ms，再写 0x00

03 **装 DMP 固件**
失败就重试，成功后才能开 FIFO

04 **静止采 200 次算零偏**
约 1 秒，全程车不能动

顺序反了会怎样：先开 DMP 再校准，第一秒的数据就带着零偏被积进去

Q30 是定点数，先除再算

```
#define q30  1073741824.0f        // 2^30
q0 = quat[0] / q30;  q1 = quat[1] / q30;
q2 = quat[2] / q30;  q3 = quat[3] / q30;

pitch = asin(-2*q1*q3 + 2*q0*q2) * 57.3f;
roll  = atan2(2*q2*q3 + 2*q0*q1,
              -2*q1*q1 - 2*q2*q2 + 1) * 57.3f;
yaw   = atan2(2*(q1*q2 + q0*q3),
              q0*q0 + q1*q1 - q2*q2 - q3*q3) * 57.3f;
```

DMP 给的是整数

放大 2^{30} 倍，不除会得到天文数字

57.3 = 180 / π

弧度转角度

不用归一化

DMP 输出的模长已经是 1

四元数避开万向节死锁：Pitch 到 ±90° 时，欧拉角分不清 Roll 和 Yaw

角度符号，别搞反

三个姿态角的正方向

动作	角	符号
抬头（机头上扬）	Pitch	正
点头（机头下压）	Pitch	负
右侧倾斜	Roll	正
左转（逆时针俯视）	Yaw	正

屏幕上的正负和“右手定则”一致：拇指指向轴的正方向，四指弯曲方向为正

Yaw 为什么漂，怎么压

Yaw 漂移的对策

对策	效果	说明
零偏校准	明显减缓	从每分钟 10° 压到 1° 量级
温度补偿	有一定效果	零偏随温度变，可建温度-零偏表
加磁力计	根本解决	靠地磁提供绝对航向
编码器 / 轮速辅助	短期很有效	麦轮车本身有编码器，用里程计修正
动作前清零	实用土办法	只关心相对转角，漂移被关在几秒内

重力只能给 Roll / Pitch 提供参考，Z 轴在重力场里没有绝对方向——这是物理限制

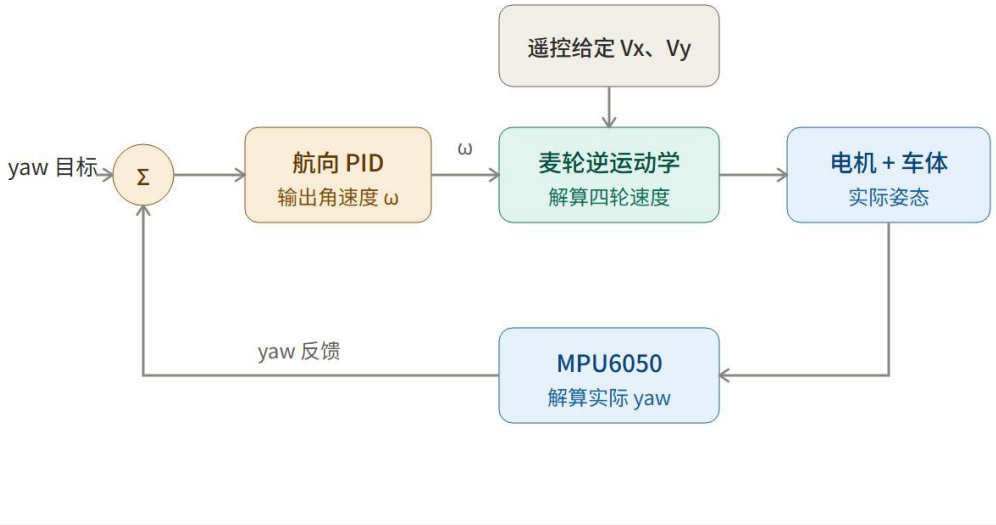
其他融合算法

常见姿态融合算法对比

算法	运算量	特点
互补滤波	最小，几次乘法	够用，参数直观
Mahony（改进互补滤波）	小	PI 修正，收敛快
Madgwick（梯度下降法）	中	梯度下降，低采样率也稳
卡尔曼	大，矩阵运算	最优估计，调参最难

> 本课用 DMP（硬件融合）；想自己写，从互补滤波起步性价比最高

把姿态用起来：航向闭环



小车航向闭环

为什么麦轮车需要

横移、斜走时四轮受力不一致，车会自己转头

偏了就拉回来

Yaw 反馈进 PID，输出左右轮差速

开环直行做不到

闭环之后“走直线”才有可能

程序分层架构

C10B 姿态部分的分层

层级	文件	职责
应用层	main.c	调度 100 Hz 时基，读角度送显示
驱动层	MPU6050.c	封装成“读一次角度”
通讯层	inv_mpu.c	寄存器、FIFO、DMP 使能
固件层	inv_mpu_dmp_motion_driver.c	3062 B 固件与解算
板级层	bsp_siic.c	软件 I²C 四个基本动作

单片机只做两件事：I2C 取包、四元数换算成 Roll/Pitch/Yaw，最后送 OLED 显示。

> 换芯片只动板级那几个宏：读、写、延时、打印

程序分层架构

应用层	main.c main() ——▶ MPU6050_initialize() / DMP_Init() EXTI 中断 ——▶ Read_DMP() ——▶ Roll/Pitch/Yaw ——▶ OLED / UART5
驱动层	MPU6050.c / MPU6050.h （WHEELTEC 二次封装） IICwriteBits / IICwriteBit / IICreadBytes / i2cRead MPU6050_initialize / DMP_Init / Read_DMP / Read_Temperature

程序分层架构

通讯层	inv_mpu.c （InvenSense 官方驱动） mpu_init / mpu_set_sensors / mpu_configure_fifo mpu_set_sample_rate / mpu_write_mem / mpu_load_firmware mpu_set_dmp_state / mpu_read_fifo_stream / mpu_run_self_test
固件层	inv_mpu_dmp_motion_driver.c dmp_memory[3062] ← DMP 固件镜像（写进芯片 DMP 内存） dmp_load_motion_driver_firmware / dmp_set_orientation dmp_enable_feature / dmp_set_fifo_rate / dmp_read_fifo

程序分层架构

板级层	bsp_siic.c （软件模拟 I2C，PB14=SCL / PB15=SDA） Start/Stop/WaitAck/SendByte/ReadByte IIC_Mem_Read / IIC_Mem_Write ——▶ User_sIICDev 函数指针表

一个中断，五个任务

DMP 每 10 ms 出包

INT 引脚（PB9）下降沿触发

读一包姿态

dmp_read_fifo → 四元数 → 欧拉角

每 5 次发一次超声波

50 ms 一次 15 μs 高脉冲

每 5 次给底盘发一帧

20 Hz，12 字节，UART5

每 100 次查一次 K210

中断里只做标记和搬数据，超时上限约 1.2 ms；阻塞发送是这里最大的隐患

中断里的时间预算

100 Hz 中断的耗时估算

任务	频率	耗时
读 DMP 一包	100 Hz	约 0.9 ms（I²C 32 字节）
触发超声波	20 Hz	几个 µs
读温度	0.2 Hz	约 0.2 ms
发一帧给 C50C	20 Hz	约 1.04 ms（12 字节）

合计接近 1.2 ms / 10 ms，占 12%；再塞任务就会丢中断

实验三：从裸数据到角度

01

读 ID 与原始值

先把加速度、陀螺的 14 字节读出来显示

02

看静止时的零偏

陀螺三轴应接近 0，偏大就校准

03

跑通 DMP

屏上出现会随车转动的三个角

04

验证方向

抬头 Pitch 变大，右转 Yaw 变大

实验异常分析

现象与原因对照

现象	最可能的原因	动作
WHO_AM_I 读不到	SDA/SCL 接反、没上拉	量电平，确认开漏
全是 0xFF	总线被拉低卡死	检查哪根线短路
角度一直 0	DMP 固件没装成功	看加载日志/返回值
角度乱跳	减震不够、DLPF 太宽	拧紧 + 降到 42 Hz
Yaw 转一圈不回来	正常漂移	做零偏校准，或只做相对角
一上电就重启	ID 不对触发 NVIC_Reset	先查线再上电

第二讲小结：四个结论

六轴是两种传感器

一个看方向，一个记过程，都不完美

DMP 省掉全部数学

代价是只能拿到四元数

Yaw 一定漂

要么加磁力计，要么只做相对转角

数据乱跳先查机械

减震、刚性固定比改代码有效

下一讲换到串口：这些数据怎么变成一个系统里的帧

CHAPTER 03

03

串口通讯

把前面这些串成一个能对话的系统

本讲内容

- 电气标准与传输方向
- 一帧的解剖
- HAL 初始化与三种收发
- 三层校验与排错

本讲要get到什么

说清一帧的每一位

空闲、起始、数据、校验、停止

波特率与校验位

位时间、误差 2%、奇偶各一遍

会配 HAL 与 DMA

七个参数、USART3 重映射、DMA1_Ch3

能独立排错

乱码先查波特率，不通先查线序

> 串口是唯一“跨模块”的一讲：C10B 与 C50C、K210 交互全靠它

三课时怎么排

课时一：协议基础

电气标准、同步异步、一帧解剖、波特率与校验

课时二：STM32 实现

七个参数、MspInit、DMA 通道、三种收发

课时三：协议与调试

BCC 组帧、两块板的报文、排错与联调

这一讲最像工程

同样的代码，参数错一位就是全盘乱码

> 串口是唯一跨模块的一讲：C10B、C50C、K210 靠它连成一个系统

为什么是串口

线少

三根就能全双工，布线简单

抗干扰

差分传输能拉到千米级

所有芯片都带

从 8 位单片机到服务器

调试第一手段

一行打印胜过十个断点

常见误解：串行比并行慢。USB 3.x、PCIe、千兆网全是串行——一位跑得够快就行

串行与并行

并行

一次传多位，线多

引脚成本高、串扰大

各线时序偏移难对齐

距离一长就崩

串行

一位一位排队传，线少

差分信号抗干扰强

靠高波特率补速度

现代高速总线全是它

不是“串行慢”，是“并行在高速下无法保持同步”——这是物理决定的

四种标准

四种电气标准对比

标准	电平与方式	拓扑	距离
TTL	单端，0 V / 3.3–5 V	点对点	板内，几十厘米
RS232	±3 … ±15 V，逻辑反相	点对点	约 15 m
RS485	差分，两线压差	总线，32 节点起	约 1200 m
RS422	差分，四线	一主多收，全双工	约 1200 m

RS232 是“反的”且高压：TTL 直连会烧，必须过 MAX3232 这类芯片

能直连的前提是共地

TTL 直连只适合板内

几十厘米，双方共地，3.3 V 对 3.3 V

不共地会怎样

电平没有共同参考，收到的全是 0xFF 或乱码

长线与干扰环境用差分

RS485 两根线取差值，噪声互相抵消

接线上最容易错的两件事：不共地、TX 接 TX。先查这两条再怀疑代码

单工、半双工、全双工

三种传输方向

类型	定义	典型例子
单工	只能单向流，没有反向通路	广播、红外遥控
半双工	两端都能发，但同一时刻只能一方占用	RS485、对讲机
全双工	两端可同时收发，互不干扰	UART 的 TX / RX、电话

UART 天生全双工：TX 和 RX 是两根独立线，发的时候可以同时收

同步与异步通讯

同步与异步六维对比

维度	同步通讯	异步通讯
时钟线	有独立时钟（SCLK / CK）	没有，靠波特率约定
帧边界	由时钟边沿界定，可连续流	由起止位界定
效率	几乎 100%	8N1 下开销 20%
硬件	至少 3 根线	3 根即可全双工
典型协议	SPI、I²C、USART 同步模式	UART、RS232、RS485

异步：没有时钟，怎么对齐

01 **线路空闲保持高电平**
接收方盯着这根线，等一个下降沿

02 **起始位：拉低一个位时间**
接收方检测到下降沿，位计时器清零

03 **按约定节拍逐位采样**
每位采三次取多数，抗一点噪声

04 **停止位：拉高一个位时间**
保证线路回高，下一帧才认得出起始

波特率

每帧只同步一次，所以帧必须短：误差正比于帧长，这就是 8 位数据位的由来

帧同步：先找到帧头

空闲时收到的字节是随机的

可能是上一帧尾巴，也可能是噪声

只在看到帧头时开始计数

0xCC 一出现就当新帧开始

攒满长度再做两项检查

帧尾对不对、BCC 对不对

错一帧就静默丢弃

等下一个帧头，不重传

> 视觉数据是连续刷新的，丢一帧对控制几乎没影响，重传反而更复杂

一帧由五段组成

UART 一帧的字段构成

字段	长度	电平	作用
空闲位	不定长	高 1	无数据时的线路状态
起始位	1 位	低 0	通知对方并同步位计时器
数据位	5 – 9 位	视数据	有效数据，低位先发
校验位	0 或 1 位	视方式	可选，奇偶校验
停止位	1 / 1.5 / 2 位	高 1	标志结束，线路回空闲

8N1 = 8 位数据 + 无校验 + 1 位停止，一帧 10 位，其中只有 8 位有效

8N1发送0x55为例

发送 0x55 时的线上位序

位置	空闲	起始	D0	D1	D2	D3	D4	D5	D6	D7	停止
电平	1	0	1	0	1	0	1	0	1	0	1

0x55 = 0101 0101：低位先发，所以线上看起来是“反”的——用示波器看会以为错

用逻辑分析仪看一帧

探头接 TX 与 GND

单人调试时最省事的工具

触发设在下降沿

起始位就是一个下降沿

量一位的宽度反推波特率

8.68 μs 对应 115200

数 10 个位宽就是一帧

起始 + 8 数据 + 停止

> 先看波形再看代码：波形不对，代码写得再漂亮也没用

波特率：位时间与帧耗时

常见波特率下的位时间

关系	公式	9600	115200
位时间	1 / 波特率	104.17 µs	8.68 µs
一帧耗时（8N1）	10 / 波特率	1.0417 ms	86.8 µs

波特率的定义是每秒传输的符号数。在 UART 中一个符号就是一位，所以波特率等于位速率（bit/s）。

为什么误差必须小于 2%

接收方在每位中间采样

偏差不会自己纠正，只会逐位累积

一帧 10 位就累积 10 倍

第 10 位最容易采错

累计偏差不能超半个位宽

换算下来相对误差约 2%

所以波特率是标准档位

9600 / 115200 大家都用，才好对齐

两边晶振不同源时，优先降波特率：慢一点但稳，比快而错好

奇偶校验怎么算

0x55（4 个 1）的校验位演算

校验方式	规则	校验位	校验后 1 的总数
无校验 None	不添加校验位	无	4（偶数）
偶校验 Even	使 1 的总数为偶数	0	4 + 0 = 4
奇校验 Odd	使 1 的总数为奇数	1	4 + 1 = 5

再看 0x57（5 个 1）：偶校验填 1，奇校验填 0——先数 1，再看要凑成奇还是偶

BCC：整帧异或，几行就够

```
static uint8_t calculateBCC(uint8_t *buf, uint8_t len)
{
    uint8_t bcc = 0;
    for (uint8_t i = 0; i < len; i++)
        bcc ^= buf[i];
    return bcc;
}

/* 发送：算前 n-1 字节，存进最后一个字节 */
frame.bcc = calculateBCC((uint8_t *)&frame,
                         sizeof(frame) - 1);
```

异或的好处

算得快，收发对称

坏处

两字节同时错会互相抵消

所以还要看帧尾和长度

三重条件一起判，误判率极低

常见帧格式速查

常见帧格式

写法	数据位	校验	停止位	帧总长	适用场合
8N1	**8**	**无**	**1**	**10 位**	**最常用：模块互联、打印**
8E1	8	偶	1	11 位	需要弱校验的工业设备
8O1	8	奇	1	11 位	约定为奇校验的设备
7E1	7	偶	1	10 位	老设备，只传 ASCII

四项参数必须逐项一致：波特率、数据位、校验、停止位，错一项就是乱码

USART 与 UART 的区别

USART 多一个 “S”

Universal Synchronous / Asynchronous

同步模式要多一根 CK 线

由它输出时钟，接收方跟着走

UART 只能异步

F103 上 UART4 / UART5 就是这种

F103 一般有 3 ~ 5 个

USART1/2/3，大封装带 UART4/5

本课全用异步模式，所以 USART 和 UART 用法上没有区别

挂在哪条总线，决定上限

F103 串口外设的总线归属

外设	挂载总线	总线最高频率
USART1	APB2	72 MHz
USART2 / USART3	APB1	36 MHz
UART4 / UART5	APB1	36 MHz

同样配 115200，USART1 的上限是 USART2 的两倍——波特率是从总线时钟分出来的

C10B 上的三个串口

本工程用到的串口与引脚

用途	外设	引脚	方式
调试打印	USART1	PA9 / PA10	阻塞发送
接 K210 视觉	USART3	PC10 / PC11	DMA + IDLE 收
接 C50C 底盘	UART5	PC12 / PD2	20 Hz 阻塞发

大部分引脚都是复用的，要注意设置。

注意交叉：本方 TX 接对方 RX。原理图上标的“串口 4”与程序不一致，以代码为准

硬件替你做了什么

01 **写 DR 就发**
起始位、停止位由硬件自动补上

02 **移位寄存器逐位推**
位时序由波特率寄存器保证

03 **接收三取二采样**
每位采三次，取多数值

04 **凑满一帧置 RXNE**
通知程序“数据可以取了”

所以协议细节不用你算，你要做的是把参数填对、把数据搬走

七个关键参数

CubeMX 串口配置的七个参数

参数	常用值	含义与影响
Baud Rate	115200	必须与对方一致，错一点就是乱码
Word Length	8 Bits	含校验位：8 数据 + 1 校验要选 9
Parity	None	无 / 偶 / 奇，与对方一致
Stop Bits	1	1 / 1.5 / 2，绝大多数用 1
Mode	Asynchronous	本课固定异步
Flow Control	Disabled	RTS/CTS 握手，一般禁用
NVIC	按需	用中断收发就必须勾

七个参数，收发双方要一致！

串口初始化

01 **开时钟**
GPIO 在 AHB，USART1 在 APB2，其余在 APB1

02 **配复用与引脚**
AF 编号别写错，TX 推挽、RX 浮空/上拉

03 **配 NVIC**
要中断就设优先级并使能

04 **配参数与使能**
8N1 + RXNE 中断 + USART_Cmd

> 六步里最容易漏的是“开 AFIO 时钟”——重映射会静默失效

初始化函数只填参数

```
UART_HandleTypeDef huart3;   // 全局句柄

huart3.Instance    = USART3;              // APB1, 36 MHz
huart3.Init.BaudRate   = 115200;
huart3.Init.WordLength = UART_WORDLENGTH_8B;
huart3.Init.StopBits   = UART_STOPBITS_1;
huart3.Init.Parity     = UART_PARITY_NONE;
huart3.Init.Mode       = UART_MODE_TX_RX;
huart3.Init.HwFlowCtl  = UART_HWCONTROL_NONE;
huart3.Init.OverSampling = UART_OVERSAMPLING_16;
HAL_UART_Init(&huart3);                   // 失败先查时钟
```

usart.c

HAL_UART_Init 做什么

按总线时钟算出波特率分频，写进寄存器

失败常见原因

AFIO 时钟没开，重映射无效

句柄是全局的

后面收发都要用到它

MspInit：引脚与重映射

```
__HAL_RCC_USART3_CLK_ENABLE();
__HAL_RCC_GPIOC_CLK_ENABLE();

GPIO_InitStruct.Pin  = GPIO_PIN_10;   // TX
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

GPIO_InitStruct.Pin  = GPIO_PIN_11;   // RX
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

__HAL_AFIO_REMAP_USART3_PARTIAL();
```

默认引脚是 PB10/PB11

本板走 PC10/PC11，属部分重映射

少这一句会怎样

串口毫无反应，代码看着全对

RX 建议上拉

浮空输入 + 长杜邦线容易误码

中断优先级怎么排

本工程的建议优先级

中断	抢占优先级	理由
MPU6050 EXTI	4	100 Hz 时基，丢一拍全乱
USART3 IDLE/DMA	5	视觉数据，允许略等
TIM2 捕获	5	超声回波，精度靠硬件锁存
UART5 发送	—	阻塞式，不占中断
SysTick	15	最低，只做计时

数字越小越优先；同级之间按子优先级，别让两个高优先级互相抢占

为什么必须是 DMA1_Ch3

STM32F1 的 DMA 请求是硬连线

不能随便挑通道，查手册映射表

Channel2 给 USART3_TX

Channel3 给 USART3_RX

写错通道什么也不报

DMA 永远等不到请求，接收一直超时

还要 __HAL_LINKDMA（）

把 DMA 句柄挂到 uart->hdmarx 上

> 这一步只能背下来：USART3_RX ↔ DMA1_Channel3，F103 全系列都一样

调用顺序不能反

```
HAL_Init();                     // 1. HAL 与 SysTick
SystemClock_Config();           // 2. 时钟：波特率靠它算
MX_GPIO_Init();                 // 3. 先配引脚
MX_DMA_Init();                  // 4. DMA1 时钟 + 通道
MX_UART5_Init();                // 5. C50C，只发
MX_USART3_UART_Init();          // 6. K210，DMA 收
HAL_UARTEx_ReceiveToIdle_DMA(&huart3, k210_buffer, 30);
```

时钟必须最先

波特率分频依赖 APB1 / APB2 频率

引脚必须先于串口

否则复用功能没生效

DMA 必须先于串口

MspInit 里要链接 DMA 句柄

阻塞式：最简单，也最危险

```
uint8_t tx_buf[] = "Hello, UART!";
uint8_t rx_byte;

/* 阻塞发送：把 14 个字节全部发完才返回，超时 100 ms */
HAL_UART_Transmit(&huart1, tx_buf, 14, 100);
// 14 字节约 1.2 ms @115200，通常可接受

/* 阻塞接收：等到收到 1 个字节，或等待超过 100 ms 后超时返回 */
HAL_UART_Receive(&huart1, &rx_byte, 1, 100);
// 对方不发，这一行就卡满 100 ms
```

timeout 是“最多等多久”

不是“必须等多久”

HAL_MAX_DELAY 很危险

对方不发就永远停在这

适用场景

上电打印、低频调试输出

带看门狗的系统里，阻塞接收会直接导致复位

阻塞到底卡多久

阻塞收发的时间代价

动作	数据量	115200 下耗时
发一帧 12 字节	12 B	约 1.04 ms
发 100 字节	100 B	约 8.7 ms
发 12 字节 @9600	12 B	约 12.5 ms
收 1 字节（超时）	—	最多等满 timeout

> 发送的耗时可以预估，接收的耗时不可预估——这是阻塞最坑的地方

中断式：把等待交给硬件

```
uint8_t rx_byte;
volatile uint8_t rx_flag = 0;   // 中断里会改
/* 在主循环开始前，先开启一次中断接收 */
HAL_UART_Receive_IT(&huart1, &rx_byte, 1);

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *h)
{
    if (h->Instance == USART1) rx_flag = 1;
    HAL_UART_Receive_IT(&huart1, &rx_byte, 1);  // 必须重开
}
```

只收一个字节就回调

因为接收长度写的是 1

忘了重开就只收第一帧

这是最高频的 bug

回调里别干活

不打印、不延时、不解析

字节进环形缓冲，中断立刻退出

```
typedef struct {
    uint8_t buf[64];
    volatile uint16_t head, tail;
} ring_t;

static inline void ring_put(ring_t *r, uint8_t b)
{
    uint16_t n = (r->head + 1) % 64;
    if (n != r->tail) {           // 满了就丢，绝不阻塞
        r->buf[r->head] = b;
        r->head = n;
    }
}
```

中断里只入队

解析放主循环，中断时间恒定

头尾指针用 volatile

否则编译器会优化掉读取

满了丢新数据

比覆盖旧数据更容易发现问题

DMA + IDLE：不定长的正解

```
uint8_t rx_buf[64];

/* DMA 发送：函数立刻返回，后续搬运由 DMA 控制器完成 */
HAL_UART_Transmit_DMA(&huart1, tx_buf, sizeof(tx_buf) - 1);

/* DMA 接收（定长）：收满 64 字节后触发回调 */
HAL_UART_Receive_DMA(&huart1, rx_buf, 64);   // 定长

// 不定长：DMA 搬字节，IDLE 判帧尾
HAL_UARTEx_ReceiveToIdle_DMA(&huart3, k210_buffer, 30);
//收到空闲了或者收满30触发回调
```

```
// 空闲中断回调里拿到这一包的长度
void HAL_UARTEx_RxEventCallback(...) { ... }
```

DMA 只搬不解包

CPU 占用几乎为零

IDLE 判断帧结束

总线超过一个位时间没动静

坑：缓冲区被覆盖

收到就拷贝出来处理，或用双缓冲

IDLE 判帧的三种意外

一次收到两帧

K210 连发时可能是 26 字节，要逐字节喂状态机

收到半包

干扰导致提前空闲，等超时后丢弃

永远等不到 IDLE

对方持续发，改判断“帧头 + 长度”

重启别忘了

NORMAL 模式下每次回调都要重新启动 DMA

> 回调里写 for (i = 0; i < Size; i++) 逐字节处理，是这里的关键

三种方式怎么选

阻塞、中断、DMA 对比

维度	阻塞式	中断式	DMA
CPU 占用	全程空等	每字节一次中断	几乎为零
实时性	差，卡主循环	好	最好
适用场景	上电日志、低频打印	单字符命令交互	不定长收包、高吞吐
典型 API	Transmit / Receive	Transmit_IT / Receive_IT	ReceiveToIdle_DMA

选型原则：先看数据量和实时性，再看实现成本。没有最好，只有最合适

115200 到底能传多少

带宽与占用核算

项目	数值	来源
一位时间	8.68 µs	1 / 115200
一帧 8N1	86.8 µs	10 位
理论最多帧数	约 11500 帧/秒	1 s ÷ 86.8 µs
本工程占用	约 10%	20 Hz × 12 字节

算清楚就知道还有多少余量：20 Hz 上报远远没到瓶颈

数据校验：奇偶位之外，还有两层

数据校验的三个层级

层级	手段	能对付什么	局限
位级	奇偶校验位	单个位翻转	只能检奇数个错，不能纠
帧级	校验和 / 异或和	整帧若干字节出错	两字节同时错会抵消
帧级	CRC16 / CRC32	连片突发错误	计算稍复杂，能力最强

奇偶位只提高一位错的检出率；真正保证整包完整的是帧级校验

K210 → C10B：13 字节帧

串口 3 通讯协议

偏移	字段	说明
0	Head	固定 0xCC
1 ~ 4	Cam_W / Cam_H	图像宽高，如 320×240
5 ~ 6	cx	目标中心 X（像素）
7 ~ 8	cy	目标中心 Y（像素）
9 ~ 10	size	目标面积（像素数）
11	BCC	字节 0~10 逐字节异或
12	End	固定 0xDD

定长 + 帧头帧尾 + 异或，是本项目里所有报文的统一套路

K210发的一帧真实数据长什么样

例：320×240、目标 (160,120)、面积 100

字段	取值	线上字节
帧头	0xCC	CC
图像宽高	320 × 240	40 01 F0 00
目标中心	(160, 120)	A0 00 78 00
目标面积	100	64 00
校验 / 帧尾	0xC1 / 0xDD	C1 DD

小端模式

逐字节异或：CC^40^01^F0^00^A0^00^78^00^64^00 = C1，与第 11 字节一致

C10B → C50C：12 字节帧

串口 5 通讯协议

偏移	字段	说明
0 ~ 1	Head1 / Head2	0xCC 0xDD 双帧头
2	k210_alive	1 = 1 秒内收到过视觉数据
3 ~ 4	k210_cx	目标中心 X，小端
5 ~ 6	k210_size	目标面积，小端
7 ~ 8	distance	距离，单位 mm
9 ~ 10	temperature	温度 ×100（26.50 ℃ → 2650）
11	BCC	字节 0~10 异或

20 Hz 发送：复用 100 Hz 中断，每 5 次发一帧，省一个硬件定时器

逐字节组帧的状态机

```
recv_data[recv_counts] = recv;          // 先写入
if (recv == 0xCC || recv_counts > 0)    // 找帧头或已在帧内
    recv_counts++;
else
    recv_counts = 0;                // 空闲态收到杂字节，丢弃

if (recv_counts == 13) {                // 收满一帧
    recv_counts = 0;
    if (recv_data[12] == 0xDD &&
        recv_data[11] == calculateBCC(recv_data, 11))
        memcpy(&k210, recv_data, 13);  // 两项都过才生效
}
```

先写再判是“擦边安全”

长度固定时刚好不越界

两项检查缺一不可

帧尾过、BCC 不过，说明数据被改过

失败静默丢弃

等下一个 0xCC，不做重传

串口通讯故障排查

串口故障排查表

现象	最常见原因	排查动作
全是乱码	波特率不一致 / 时钟配错	核参数，量一个位的实际宽度
完全没有反应	TX/RX 没交叉、没共地	确认交叉与 GND，关掉占用端口的软件
只收到第一个字节	回调里没重开接收	检查是否又调了 Receive_IT
偶发丢数据	没缓冲，新数据盖旧数据	上环形缓冲，或改 DMA + IDLE
上电先来一个乱码	上电抖动被当成起始位	丢弃开机第一个字节
收到全是 0 或 0xFF	电平标准不匹配	TTL 直连了 RS232，加转换芯片

乱码先查波特率，不通先查线序和共地——这两条覆盖绝大多数故障

上机排错，按这个顺序走

01 **看波形**
逻辑分析仪抓 TX，有没有起始位、位宽对不对

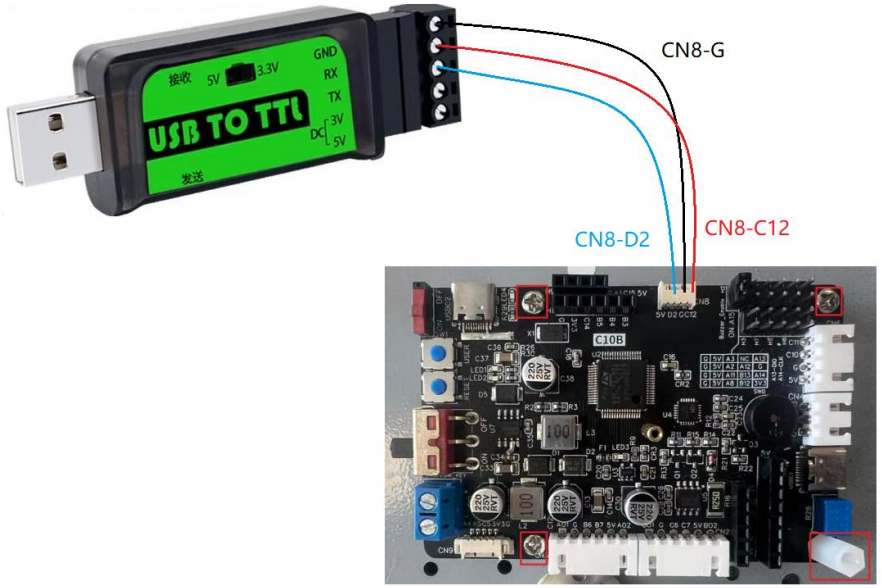
02 **量线序**
TX 对 RX、GND 对 GND，用万用表通断档

03 **对参数**
波特率、数据位、校验、停止位逐项一致

04 **改代码**
前三条都对还不行，才是缓冲区/回调的问题

顺序反了最费时间：改半天代码，最后发现是线序接反

串口调试



C10B 串口调试接线

USB 转串口，电平选 3.3 V

接错 5 V 电平可能打坏 IO

RX / TX 要交叉

同名接同名是新手最常犯的错

助手里参数要一致

端口号在设备管理器里查

串口助手


WHEELTEC—串口助手
AC
CC DD 00 00 00 00 00 04 01 4C 0A
52
CC DD 00 00 00 00 00 04 01 4C 0A
52
CC DD 00 00 00 00 00 04 01 4C 0A
52
CC DD 00 00 00 00 00 04 01 4C 0A
52
CC DD 00 00 00 00 00 04 01 4C 0A
52
CC DD 00 00 00 00 00 04 01 4C 0A
52
CC DD 00 00 00 00 00 04 01 4C 0A
52
3
串口配置
端口号:
COM4:USB-SERIAL CH
波特率:
115200
停止位:
1
数据位:
8
校验位:
无校验
保存窗口
1
清除接收
16进制显示
时间戳:
0
ms
定时发送
周期:
0
ms
Disconnect
打开串口
2
数据发送
发送
清除发送
16进制发送
发送新行
R:71588
S:0
当前时间: 20:59:34
【WHEELTEC官方旗舰店】

两个工具，一个目的

串口助手：看内容

参数与单片机一致，RX/TX 交叉，电平选 3.3 V

逻辑分析仪：看时序

能抓到波特率错误、帧粘连、位宽畸变

先看时序再看内容

内容乱码时，波形往往已经告诉你原因

USB 转串口模块电平一定选 3.3 V，5 V 电平可能打坏 IO

实验四：三块板联调

01 **单板自环**
TX 接 RX，发什么收什么

02 **接串口助手**
先确认 115200 能打印一行字

03 **接 K210**
看 cx / cy / size 有没有在变

04 **接 C50C**
底盘收到 20 Hz 报文，车能动

本讲最常见的八个毛病

表 7.2　现象与原因对照

现象	最可能的原因	动作
全乱码	波特率 / 时钟配错	量一位宽度反推
一个都收不到	没共地、TX/RX 没交叉	万用表打通断
只收到第一帧	回调没重开接收	在回调末尾再调一次
收到半包	IDLE 提前触发	加长度与 BCC 双重判断
数据偶尔错一位	地线太长、干扰	缩短走线，加 22~100 Ω
两块板数据不一致	小端/大端或结构体对齐	按字节拼，别直接 memcpy

课后任务：

把C10B计算的姿态数据发给C50C，并在OLED上显示

第三讲小结：五条结论

一帧 10 位，8 位有效
起始位和停止位是同步付的税

波特率是双方合同
误差 2% 以内，优先用标准档位

参数逐项一致才通
波特率、数据位、校验、停止位

收完必须重开
中断一次性，DMA 也要重启

排错有顺序
线序 → 参数 → 波形 → 代码

三讲合起来，就是一块板子从“看得见”到“说得上话”的完整链路

SUMMARY

三讲回顾

看得见 · OLED 显存与时序

感知得到 · 超声波与输入捕获

动得起来 · 蜂鸣器与舵机

站得稳 · MPU6050 与 DMP

说得上话 · UART 一帧到排错

三条工程建议

先看时序图，再写代码

先量电压，再怀疑芯片

中断里只贴标记，活留给主循环